MEDIATEK GENIO · TEST CATALOGUE

Genio Jailhouse and Zephyr **Test Catalogue**

Genio 510 and Genio 700 EVKs
Verification and validation

APPLIES TO	Genio 510 EVK (MT8370) and Genio 700 EVK (MT8390)
SOFTWARE	MediaTek Jailhouse `e7fd8796`; IoT Yocto `rity-bringup-image` with the `jailhouse` feature, Linux 6.6.152; Genio Zephyr image set rel-1.1
REVISION	1.1.1 · 2026-10-26

This catalogue describes every test that verifies and validates the Jailhouse hypervisor and Zephyr on the Genio 510 and Genio 700 EVKs: what each test does, why it is needed, what it checks, and the output of a passing run, taken from the test scripts and the board logs.

FOR REVIEW

MEDIATEK **GENIO**

Document information

Title	Genio Jailhouse and Zephyr — Test Catalogue
Document type	Test catalogue
Status	For review
Revision	1.1.1
Applies to	Genio 510 EVK (MT8370) and Genio 700 EVK (MT8390)
Software	MediaTek Jailhouse `e7fd8796`; IoT Yocto `rity-bringup-image` with the `jailhouse` feature, Linux 6.6.152; Genio Zephyr image set rel-1.1
Tests	111
Last updated	2026-10-26

REVISION HISTORY

Revision	Date	Changes
1.0	2026-10-26	First issue, for review
1.1	2026-10-26	Re-runs of W1-TWO, W1-M3-T18 and REL-03 to REL-05 (board logs 188 to 190) and the script changes of 2026-10-26 in the entries and Chapter 7; the reset cause in W1-D5/D6 and REL-10; Chapter 8, proposed coded criteria
1.1.1	2026-10-26	Public copy: the sources are cited by board log number and file name, without the paths of the internal workspace. No test, check or result changed

Contents

1 **Introduction** . . . 5
- 1.1 Purpose and scope . . . 5
- 1.2 The test entries . . . 6
- 1.3 Test set-up . . . 6
- 1.4 Results of the latest runs . . . 7
- 1.5 Sources . . . 8

2 **Regression** . . . 9
- 2.1 Overview . . . 9
- 2.2 Hypervisor start and the root cell . . . 10
- 2.3 The bare-metal inmate and Zephyr in the default cell . . . 11
- 2.4 Zephyr on a Cortex-A78 . . . 15
- 2.5 The SMP cell . . . 17
- 2.6 Audio in the AFE cells . . . 20
- 2.7 The root cell's access to shared registers . . . 23
- 2.8 Cell configuration checks and the CPUs . . . 25
- 2.9 End of the run: console, disable and re-enable . . . 26

3 **The watchdog set W1** . . . 29
- 3.1 Overview . . . 29
- 3.2 Stage M2: the watchdog unit . . . 30
- 3.3 Stage M3: reset causes, PSCI resets and faults . . . 35
- 3.4 Stage Z: Zephyr watchdog applications . . . 40
- 3.5 Stage TWO: two cells with watchdogs . . . 42
- 3.6 Stage D: the restart daemon . . . 44

4 **Release-level checks** . . . 48
- 4.1 Overview . . . 48
- 4.2 Samples and tools . . . 48
- 4.3 Boot without the overlay . . . 52
- 4.4 Real resets of the SoC . . . 53
- 4.5 Checks of specific changes . . . 55

5 **Image and flash checks** . . . 60

6 **Host-side checks** . . . 65

7 **Status of the scripts** . . . 72
- 7.1 Verdicts . . . 72
- 7.2 Open points . . . 73
- 7.3 Expected output not captured in the logs . . . 73
- 7.4 Script changes of 2026-10-26 . . . 74

8 **Appendix: proposed coded criteria** . . . 75

8.1 rity-w1.sh (Chapter 3) . . . 75
8.2 first-boot.sh (IMG-02) . . . 77
8.3 rity-precheck.sh (IMG-01) . . . 77
8.4 refine-accept.sh (IMG-03) . . . 77
8.5 Not covered . . . 77

9 **Test index** . . . 78

01
Introduction

1.1 Purpose and scope
1.2 The test entries
1.3 Test set-up
1.4 Results of the latest runs
1.5 Sources

1.1 Purpose and scope

This catalogue describes the tests that verify and validate the MediaTek Jailhouse hypervisor and its Zephyr cells on the Genio 510 and Genio 700 EVKs. For each test it states why the test is needed, what the test does, what it checks, and the output of a passing run. All of this is taken from the test scripts and from the logs of the board runs.

The tests fall into five groups. Each group is a chapter.

Chapter	Group	Identifiers	Tests	Runs on
2	Regression	R-01 to R-52	52	Both boards: every release candidate and every image change
3	Watchdog set W1	W1-M2-T01 to W1-D6	33	Both boards: every image that ships Jailhouse
4	Release-level checks	REL-01 to REL-12	12	One or both boards: every image, or once per change they cover
5	Image and flash checks	IMG-01 to IMG-04	4	Both boards: every flashed image
6	Host-side checks	HOST-01 to HOST-10	10	The build host; no board

Chapter 7 gives the status of the scripts: which ones decide pass or fail themselves, the open points, the outputs that the logs do not contain, and the script changes of 2026-10-26. Chapter 8 is an appendix of proposed coded criteria for the scripts that print only observations; it is not implemented. Chapter 9 is an index of all tests.

Out of scope:

- **Measurements that are not pass/fail gates.** These are the core-count and interrupt-latency matrix of release 1.1 and the dual Cortex-A78 experiment (board logs 143 to 150), and the eTDM loop-latency report (a report of its own).
- **Sanity and unit tests proposed for the merge requests.** They do not exist yet.

1.2 The test entries

Each test has one entry with the same fields. Checks that run the same code with different inputs share one entry, which carries all their identifiers (for example R-25 to R-34) and a table of the instances.

Field	Content
Identifier and name	A stable identifier and a short name
Purpose	The risk or requirement behind the test: what would go wrong, and for whom, if it were not checked
Procedure	What the script does, step by step: cells, images, commands on the board, the console used, the boards, the preconditions
Checks	The pass criteria as the script codes them, with their tolerances. Where a script prints observations without a verdict, the criteria of the engineering notes, and the entry says so
Expected output	The script's lines on a pass, and characteristic lines of the board logs, quoted verbatim from a passing run. `[...]` marks omitted text
Last run	Date, board, image, board log and result
Sources	The script and the logs the expected output is taken from

Where a test's output is not in the logs, the entry says "Not captured in the logs". Section 7.3 lists these cases.

Conventions:

- **Boards** are called the 510 and the 700.
- **Board logs** are cited by number: board log 187 is the record of one run, numbered in the order of the runs. Where a run has several files, the entry names the file, for example board log 187 (`hv-enable.log`). The board logs, scripts and build records are internal to MediaTek's engineering workspace.
- **Cells** are named by their configuration file without the `genio-<board>-evk-` prefix: `-zephyr-afe` is `genio-700-evk-zephyr-afe.cell` on the 700. A cell's name is the same suffix (`zephyr-afe`); the default Zephyr cell is named `zephyr`.
- **Journal times** in the daemon tests come from the board's clock, which is not set. Only the differences between them are meaningful.

1.3 Test set-up

Boards.

	Genio 510 EVK	Genio 700 EVK
SoC	MT8370	MT8390
CPUs	0-3 Cortex-A55, 4-5 Cortex-A78	0-5 Cortex-A55, 6-7 Cortex-A78
`-zephyr`, `-zephyr-afe`, `-zephyr-rpmsg`	CPU 3	CPU 3
`-zephyr-smp`, `-zephyr-afe-smp`	CPUs 2-3	CPUs 2-3
`-zephyr-a78`, `-zephyr-afe-a78`	CPU 5	CPU 7

Connections to the test host.

- **adb over USB.** Every command on the board goes through `adb shell`.
- **UART1** on a USB serial adapter (`/dev/ttyUSB0`): the console of the cells, 115200 baud, 8N1. The bare-metal `uart-demo` inmate uses 38400 baud.
- **UART0** (the Linux and U-Boot console, 921600 baud) only for the reset tests REL-07 and REL-08. The test host takes one serial adapter at a time, so UART0 replaces UART1 for those two tests.
- **Wires** on the 40-pin header: GPIO 38 to GPIO 40 (pins 22 and 18) for the GPIO loopback, and the eTDM loopbacks (I2SO1 to TDMIN, I2SO2 to I2SIN) for the audio loopback samples.

Software on the board.

- **The image** is IoT Yocto `rity-bringup-image`, built at the layer heads with `DISTRO_FEATURES jailhouse`, Linux `6.6.152-mtk+g45b3ad996aa0`. It ships the hypervisor (`/usr/lib/firmware/jailhouse.bin`), the driver module, the cell configurations (`/usr/share/jailhouse/cells/`), the restart daemon and its systemd unit. It is flashed with the `jailhouse.dtbo` overlay.
  - The **heads image** was built on 2026-10-22.
  - The **refinement-1 image** (2026-10-26) differs only in the kernel configuration: `HOTPLUG_PCI_PCIE` is off, and `MTK_JH_REMOTEPROC`, `RPMSG_CHAR` and `RPMSG_CTRL` are modules.
- **The hypervisor** is MediaTek Jailhouse `mtk-genio-dev e7fd8796`. It reports itself as `mtk-v1.0.0 (28-ge7fd8796)`, because release 1.1 is not tagged yet. The cell configurations are revision 16.
- **Zephyr** is the rel-1.1 image set: mtk-zephyr `2e9e70f0a16`, the samples at `b962ff3`, Zephyr SDK 1.0.1. The images print `Zephyr OS build 2e9e70f0a16c`.
- **Test tools**, pushed to `/root` by the scripts:
  - `regs32`, 32-bit and 8-bit register access from Linux;
  - `gicd-spi.py`, the GIC distributor state of one interrupt;
  - the bare-metal test inmates `psci-probe`, `sgi-probe` and `wdt-probe`;
  - test cell configurations: one set per board for the regression, two cells for REL-12, and the watchdog test cell `wdt-b`. The first two sets are built from sources of their own.

Preconditions common to the board tests.

- One board is connected over adb, and only one test run uses it at a time. The W1 and reset scripts stop if they do not see exactly one adb device.
- **UART1's clock.** Linux runtime-suspends `ttyS1`, which gates UART1's clock, and the bare-metal inmates do not set UART1 up. So a Zephyr run or an explicit clock-gate write must come before them: `regression.sh` opens the gate before `uart-demo`, and `rity-w1.sh` starts with a Zephyr `hello_world`.

1.4 Results of the latest runs

Tests	Genio 510	Genio 700
Regression, R-01 to R-52	52/52, 2026-10-26, refinement-1 image (log 179)	52/52, 2026-10-26, refinement-1 image (log 187)
Watchdog set W1	2026-10-22, heads image (log 166)	2026-10-22, heads image (log 157); W1-TWO and W1-M3-T18 2026-10-26, refinement-1 image (logs 188, 189)
REL-01 rpmsg sample	2026-10-26 (log 178)	2026-10-26 (log 186)

Tests	Genio 510	Genio 700
REL-02 `genio-inmate.sh`	2026-10-22 (log 168)	2026-10-22 (log 159)
REL-03 to REL-05	–	2026-10-26, refinement-1 image (log 190)
REL-06 no overlay	2026-10-22 (log 169)	2026-10-22 (log 160)
REL-07, REL-08 resets	2026-10-22 (logs 173, 172)	2026-10-26 (logs 181, 182)
REL-09 PCIe rebind	–	2026-10-12 (logs 140, 141)
REL-10 `SYSTEM_RESET2`	2026-10-12 (logs 135 to 137)	–
REL-11, REL-12	–	2026-10-12 (logs 125, 126)
IMG-01 to IMG-03	2026-10-26 (logs 175, 177, 178)	2026-10-26 (logs 183, 185, 186)
IMG-04 permissions	2026-10-22, on the board (log 174)	2026-10-22, the image read on the host (log 174)
HOST-01 to HOST-10	2026-10-21 to 2026-10-26, on the build host	

All latest results are passes. In the W1 runs of 2026-10-22 (logs 157 and 166), the second case of W1-TWO did not reach the watchdog because of a script error; the case passed on the 700 when run again on 2026-10-26 (log 188). Section 7.4 lists the script changes of that day.

1.5 Sources

- **Scripts**: `regression.sh`, the scripts of the other tests, and the test inmates and tools, cited by file name.
- **Board logs**, cited by number.
- **Engineering notes.** Some entries cite these, without file names:
  - *the W1 milestone notes*: the notes on watchdog milestones 2 and 3, and on the Genio 700 watchdog run, all of 2026-10-12;
  - *the RITY move notes*: the board-run notes of the IoT Yocto integration on each board, 2026-10-22, with the additions and the correction of 2026-10-26;
  - *the script-fix record*: the record of the test-script changes of 2026-10-26, one row per catalogue item;
  - *the coded-criteria proposal*: the proposal of 2026-10-26 for pass/fail criteria in the scripts that print only observations (Chapter 8). Their file names are in the internal copy of this catalogue.
- **Build records**: the IoT Yocto integration record, and the record of the Zephyr image set.

02
Regression

2.1 Overview
2.2 Hypervisor start and the root cell
2.3 The bare-metal inmate and Zephyr in the default cell
2.4 Zephyr on a Cortex-A78
2.5 The SMP cell
2.6 Audio in the AFE cells
2.7 The root cell's access to shared registers
2.8 Cell configuration checks and the CPUs
2.9 End of the run: console, disable and re-enable

2.1 Overview

Script. `regression.sh <510|700>` runs on the test host. It drives the board over adb and reads UART1, and it uses the Zephyr images of the build root `ZB` (the rel-1.1 set here). It prints one `PASS` or `FAIL` line per check, then a summary: `=== 52 passed, 0 failed`. It exits non-zero if any check fails. A full run takes about 15 minutes.

Logs. A run is one board log, with these files:

- one log per Zephyr run (`<name>.log`);
- `uart1.log`, the UART1 capture of the whole run;
- the hypervisor console at enable (`hv-enable.log`), during the run (`hv-run.log`) and after the re-enable (`hv-end.log`);
- the pin state before enable (`pinconf-native.txt`, `pins-native.txt`).

The summary printed by the script is kept as a separate board log, for example board log 187.

Script revision. The script was revised on 2026-10-26, after the latest runs, without changing any check:

- each Zephyr run's unfiltered UART output is kept as `<name>.raw`;
- the refusal messages of R-45 and R-46 become the detail of their lines;
- R-42's comment is corrected.

The line numbers in the Sources fields refer to the revised script.

How the 52 checks are counted. The script has 34 `result` lines. 28 run once. 6 run in loops, giving 2, 2, 2, 2, 10 and 6 checks. Loop instances of one code path share an entry: R-07 and R-08, R-10 and R-11, R-14 and R-15, R-20 and R-21, R-25 to R-34, and R-35 to R-40. Some checks repeat an action inside one check: R-06, R-13, R-19, R-22 and R-48. Each of them counts once.

The common Zephyr run. Most checks run one Zephyr image in one cell with the script's function `run_zephyr`:

1. Push the image to `/root/sample.bin` and destroy every non-root cell.
2. `jailhouse cell create`, `jailhouse cell load <name> /root/sample.bin -a 0x8000`, `jailhouse cell start`.
3. Wait the given time. The UART1 output since the start goes to `<name>.log`, reduced to printable strings of at least four characters.
4. **The run fails if a command fails, or if the cell is not `running` at the end.** Then the cell is destroyed.

So every Zephyr check below also requires the cell to be still running at the end of its time window. Checks timed on the host use `run_zephyr_host` instead. There, a host script owns the serial port. The cell is started 2 s after the host script, and destroyed when the script ends.

2.2 Hypervisor start and the root cell

R-01 Hypervisor enable

Purpose. The hypervisor must load with the board's root cell, take over every CPU, and bring up the MediaTek units that mediate the shared clock, EINT, GPIO and watchdog registers. Every other test depends on this. A failure here means that the image, the overlay's memory reservation or the root cell configuration is wrong.

Procedure.

1. `jailhouse disable`, in case an earlier run left the hypervisor enabled.
2. Save the pin state of the pin controller: debugfs `pinconf-pins`, and `pins` with the last digit of each pin's value masked.
3. `modprobe jailhouse; jailhouse enable genio-<board>-evk.cell`, then save `jailhouse console` as `hv-enable.log`.
4. Start the transient unit `hv-console`, which follows the hypervisor console into `/root/hv-run.log` until R-49.

Checks. `hv-enable.log` contains `Initializing unit: MediaTek` or `Initializing unit: mt8188_eint`, and `jailhouse cell list` shows a cell `running`. The detail is the version from the line `Jailhouse hypervisor ... on CPU`.

Expected output. The script's line (both boards):

```
PASS  enable (mtk-v1.0.0 (28-ge7fd8796))
```

`hv-enable.log` on the 700. The 510 lists CPUs 0 to 5.

```
Initializing Jailhouse hypervisor mtk-v1.0.0 (28-ge7fd8796) on CPU 4
Code location: 0x0000ffffc0200800
Page pool usage after early setup: mem 55/988, remap 0/131072
Initializing processors:
 CPU 4... OK
 [...]
 CPU 3... OK
Initializing unit: irqchip
Initializing unit: mt8188_clk
Initializing unit: mt8188_eint
Initializing unit: mt8188_gpio
Initializing unit: mt8188_wdt
Initializing unit: ARM SMMU v3
Initializing unit: ARM SMMU
Initializing unit: PVU IOMMU
Initializing unit: PCI
Adding virtual PCI device 00:00.0 to cell "genio-700-evk"
Page pool usage after late setup: mem 96/988, remap 532/131072
Activating hypervisor
```

Last run. Pass on both boards, 2026-10-26, refinement-1 image (board logs 179 and 187).

Sources. `regression.sh` lines 134-146; board log 187 (`hv-enable.log`).

R-02 Root pin state unchanged by the mediation

Purpose. Enabling the hypervisor must not change any pin that Linux uses. The GPIO unit takes over the pin registers, and an error in its initial state would silently reconfigure pins under running Linux drivers.

Procedure. After R-01, read `pinconf-pins` and `pins` again, with the same masking, and compare them with the copies saved before enable.

Checks. Both files are identical to the saved copies (`diff -q`).

Expected output.

```
PASS  root: pin state unchanged by mediation
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 136-137 and 148-151; board log 187 (`pinconf-native.txt`, `pins-native.txt`).

R-03 Root cell's EINT interrupt enabled

Purpose. The EINT controller's interrupt, SPI 267, stays with the root cell after the EINT unit takes over, because Linux's GPIO interrupts arrive through it. If the hypervisor left it disabled, pending or routed away, Linux would lose every GPIO interrupt.

Procedure. `python3 /root/gicd-spi.py 267` reads the GIC distributor state of SPI 267 through `/dev/mem`.

Checks. The output is exactly `enabler=1 pendr=0 activer=0 irouter=0x0`: enabled, not pending, not active, routed to CPU 0.

Expected output.

```
PASS  root: EINT SPI enabled ( enabler=1 pendr=0 activer=0 irouter=0x0)
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 125 and 153-154; `gicd-spi.py`; board log 187.

2.3 The bare-metal inmate and Zephyr in the default cell

R-04 Bare-metal inmate `uart-demo`

Purpose. Jailhouse's own bare-metal demo must run in a non-root cell, so that the hypervisor's cell life cycle and UART1 are checked without Zephyr. When a Zephyr check fails, this tells a hypervisor fault from a Zephyr fault.

Procedure.

1. Open UART1's clock gate 23: write `0x00800000` to the CG0 clear register `0x10001084`. Then write 0 to UART1's `HIGHSPEED` register `0x11001224`. `uart-demo`'s 8250 code does not reset this MediaTek-specific register, which an earlier Zephyr run may have set, and the write takes effect only while UART1's clock runs.
2. Capture UART1 at 38400 baud.
3. Create `-uart-demo`, load `/usr/share/jailhouse/inmates/uart-demo.bin` and start it. Wait 4 s, then destroy the cell.

Checks. The UART1 output since the start contains `Hello 3 from cell`.

Expected output. The script's line, and `uart1.log` of the 700 run:

```
PASS  uart-demo
```

```
Hello 1 from cell!
Hello 2 from cell!
Hello 3 from cell!
Hello 4 from cell!
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 156-171; board log 187 (`uart1.log`) lines 1-4.

R-05 Zephyr `hello_world`

Purpose. The smallest Zephyr image must boot in the default cell `-zephyr` and print on UART1. This checks the cell's CPU, memory, UART and timer setup as a Zephyr application sees them.

Procedure. `run_zephyr` with `hello_world` in `-zephyr`, 4 s.

Checks. `hello.log` contains `Hello World`, and the run succeeds (Section 2.1).

Expected output.

```
PASS  zephyr hello_world
```

`hello.log` on the 700:

```
*** Booting Zephyr OS build 2e9e70f0a16c ***
Hello World! mt8390_genio_700_evk/mt8188/a55
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 173-175; board log 187 (`hello.log`).

R-06 Ten create, start and destroy cycles

Purpose. A cell must be created and destroyed repeatedly without leaking hypervisor memory or leaving interrupt state behind. Products restart cells, both by hand and through the restart daemon.

Procedure. `run_zephyr` with `hello_world` in `-zephyr`, 1 s, ten times. Then read the last "after cell destruction" page-pool line of the hypervisor console, and SPI 267's state as in R-03.

Checks. Every cycle succeeds, and SPI 267 reads exactly as in R-03. The page pool is reported in the detail, **not checked**.

Expected output. On the 700. The 510 reports `mem 78/`.

```
PASS  zephyr: 10 create/start/destroy cycles (mem 96/ enabler=1 pendr=0 activer=0 irouter=0x0)
```

`cycle.log` holds the last cycle's output:

```
*** Booting Zephyr OS build 2e9e70f0a16c ***
Hello World! mt8390_genio_700_evk/mt8188/a55
```

Last run. Pass on both boards, 2026-10-26. The pool was 78 pages on the 510 and 96 on the 700, the same as after late setup in R-01 (board logs 179 and 187).

Sources. `regression.sh` lines 177-183; board log 187 (`cycle.log`, `hv-enable.log`).

R-07, R-08 Memory window and GPIO loopback samples

Purpose.

- **R-07:** the cell's whole RAM window must be usable, with no part shared with Linux or missing from the stage-2 mapping.
- **R-08:** a cell must drive and sense its own GPIO pins through the mediated GPIO and EINT registers, including every interrupt type. This checks the GPIO and EINT units from the cell's side.

Procedure. `run_zephyr` in `-zephyr`, 12 s each, with the Genio samples:

Test	Sample	Log
R-07	`system/memory_window`	`memory_window.log`
R-08	`gpio/loopback`, GPIO 38 driving GPIO 40	`loopback.log`

Checks.

- R-07: the log contains `MEMTEST DONE mismatches=0 -> PASS`.
- R-08: the log contains `GPIOTEST DONE passed=25 failed=0 -> PASS`.

Expected output.

```
PASS  zephyr sample system/memory_window
PASS  zephyr sample gpio/loopback
```

`memory_window.log` on the 700:

```
MEMTEST board=mt8390_genio_700_evk/mt8188/a55
MEMTEST window base=0x8000 size=0x800000 (8 MB)
MEMTEST probe array 6 MB at 0x2aad8..0x62aad4
MEMTEST .bss zeroing already touched every byte above
MEMTEST DONE mismatches=0 -> PASS
```

`loopback.log` on the 700, abridged:

```
GPIOTEST device ready, drive=pin6 sense=pin8
PASS B6-reserved-pin configure(pin0) = -22, want -EINVAL
PASS B2-configure-output configure(pin6, OUTPUT) = 0
[...]
PASS B5-rising-on-rise rising edge produced 1 event(s), want 1
[...]
PASS B7-level-high-held armed while high: ret 0, 1 event(s), want 1
[...]
PASS B4-disabled-silent after disable, 0 event(s) still arrived
GPIOTEST DONE passed=25 failed=0 -> PASS
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 185-191; board log 187 (`memory_window.log`, `loopback.log`).

R-09 Boot and timer sample

Purpose. A cell's timer must keep time: Zephyr's 5 s sleep must take 5 s on the host's clock. A wrong timer frequency, or interrupts lost on the way to the cell, would break every timed application.

Procedure. `run_zephyr_host` with `system/boot_and_timer` in `-zephyr`. The host script `uart-stamp.py` owns the serial port for 22 s. It time-stamps each line on arrival, and for every `SLEEP_START` and `SLEEP_END` pair it prints the deviation from 5 s.

Checks. The log has exactly three `iteration` lines, and every deviation is within ±100 ms.

Expected output.

```
PASS  zephyr sample system/boot_and_timer (-13.2 ms -13.3 ms +2.8 ms )
```

`boot_and_timer.log` on the 700:

```
iteration 0: -13.2 ms from 5 s
iteration 1: -13.3 ms from 5 s
iteration 2: +2.8 ms from 5 s
```

Last run. Pass on both boards, 2026-10-26. Deviations: 510 −9.0, −9.0, +7.0 ms; 700 −13.2, −13.3, +2.8 ms (board logs 179 and 187).

Sources. `regression.sh` lines 193-200; `uart-stamp.py`; board log 187 (`boot_and_timer.log`).

R-10, R-11 UART receive interrupt and runtime reconfiguration

Purpose.

- **R-10:** a cell must receive on UART1 by interrupt under load, without losing bytes. UART1's interrupt is routed to the cell by the hypervisor.
- **R-11:** a cell must be able to change UART1's baud rate at run time. The clock and divider registers it writes are its own.

Procedure. `run_zephyr_host` in `-zephyr`, with the samples' own host scripts:

Test	Sample	Host script
R-10	`uart/rx_interrupt`	`samples/uart/rx_interrupt/host/verify_echo.py`: sends 64, 2000 and 8000 bytes and compares the echo
R-11	`uart/reconfigure`	`samples/uart/reconfigure/host/check_baud.py`: listens at 115200, 9600, then 115200 again

Checks.

- R-10: the log contains `all echo tests passed`.
- R-11: the log contains `works on the wire`.

Expected output.

```
PASS  zephyr sample uart/rx_interrupt
PASS  zephyr sample uart/reconfigure
```

`rx_interrupt.log` on the 700, abridged:

```
=== test 3: 8000 bytes, sustained (interrupt load) ===
  8000 bytes in 256-byte chunks
    sent 8000 bytes, received 8000, 1.17 s (6.7 KiB/s)
    echo byte-for-byte IDENTICAL

=== firmware counters ===
    STATS rx=10065 isr=10065
    host sent 10065 bytes total (including the 0x04 request)

RESULT: all echo tests passed
```

`reconfigure.log` on the 700, the end:

```
  phase1 readable at 115200        : PASS
  phase2 readable at 9600          : PASS
  phase2 NOT readable at 115200    : PASS
  phase3 readable at 115200 again  : PASS

RESULT: runtime reconfigure works on the wire
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 202-208; board log 187 (`rx_interrupt.log`, `reconfigure.log`).

2.4 Zephyr on a Cortex-A78

The cell `-zephyr-a78` runs on a Cortex-A78: CPU 5 on the 510, CPU 7 on the 700. The tests repeat those of the default cell, because the big cores have their own CPU features, power domain and cluster.

R-12 `hello_world` on the A78

Purpose. Zephyr must boot on a big core.

Procedure. `run_zephyr` with `hello_world` in `-zephyr-a78`, 4 s.

Checks. `hello-a78.log` contains `Hello World`, and the run succeeds.

Expected output.

```
PASS  A78: zephyr hello_world
```

```
*** Booting Zephyr OS build 2e9e70f0a16c ***
Hello World! mt8390_genio_700_evk/mt8188/a55
```

The board name in the banner is that of the build target, `a55`. The same image runs on either core type.

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 210-212; board log 187 (`hello-a78.log`).

R-13 Five cycles on the A78, with the CPU back with Linux

Purpose. After a cell on a big core is destroyed, the core must go back to Linux and come online. Otherwise Linux loses a big core for good.

Procedure. `run_zephyr` with `hello_world` in `-zephyr-a78`, 1 s, five times. Then compare `/sys/devices/system/cpu/online` with `possible`.

Checks. Every cycle succeeds, and `online` equals `possible`.

Expected output. On the 700. The 510 reports `online 0-5`.

```
PASS  A78: 5 create/start/destroy cycles, CPU back with Linux (online 0-7)
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 214-219; board log 187.

R-14, R-15 Memory window and GPIO loopback on the A78

Purpose. As R-07 and R-08, on a big core.

Procedure. As R-07 and R-08, in `-zephyr-a78`. The logs are `memory_window-a78.log` (R-14) and `loopback-a78.log` (R-15).

Checks. As R-07 and R-08.

Expected output.

```
PASS  A78: zephyr sample system/memory_window
PASS  A78: zephyr sample gpio/loopback
```

The logs read as in R-07 and R-08. For example, `memory_window-a78.log` on the 700 ends with:

```
MEMTEST DONE mismatches=0 -> PASS
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 221-227; board log 187 (`memory_window-a78.log`, `loopback-a78.log`).

R-16 Boot and timer on the A78

Purpose. As R-09, on a big core.

Procedure. As R-09, in `-zephyr-a78`.

Checks. As R-09: three iterations, each within ±100 ms.

Expected output.

```
PASS  A78: zephyr sample system/boot_and_timer (+2.7 ms +2.6 ms +2.5 ms )
```

```
iteration 0: +2.7 ms from 5 s
iteration 1: +2.6 ms from 5 s
iteration 2: +2.5 ms from 5 s
```

Last run. Pass on both boards, 2026-10-26. Deviations: 510 −9.0, +6.9, +6.9 ms; 700 +2.7, +2.6, +2.5 ms (board logs 179 and 187).

Sources. `regression.sh` lines 229-234; board log 187 (`boot_and_timer-a78.log`).

R-17 UART receive interrupt on the A78

Purpose. As R-10, with UART1's interrupt routed to a big core.

Procedure. As R-10, in `-zephyr-a78`.

Checks. `rx_interrupt-a78.log` contains `all echo tests passed`.

Expected output.

```
PASS  A78: zephyr sample uart/rx_interrupt
```

The log reads as in R-10, ending with `RESULT: all echo tests passed`.

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 236-239; board log 187 (`rx_interrupt-a78.log`).

2.5 The SMP cell

The cell `-zephyr-smp` has two Cortex-A55 cores, CPUs 2 and 3. A single-core image runs on its lowest CPU. The tests cover starting the second CPU, the interrupts between the CPUs, and giving both CPUs back to Linux.

R-18 `hello_world` on the lowest CPU

Purpose. A single-core image must run unchanged in a two-core cell.

Procedure. `run_zephyr` with `hello_world` in `-zephyr-smp`, 4 s.

Checks. `hello-smp.log` contains `Hello World`, and the run succeeds.

Expected output.

```
PASS  SMP cell: zephyr hello_world on the lowest CPU
```

```
*** Booting Zephyr OS build 2e9e70f0a16c ***
Hello World! mt8390_genio_700_evk/mt8188/a55
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 241-245; board log 187 (`hello-smp.log`).

R-19 Five cycles of the SMP cell, with both CPUs back

Purpose. As R-13, for two CPUs, one of which never ran.

Procedure. `run_zephyr` with `hello_world` in `-zephyr-smp`, 1 s, five times. Then compare `online` with `possible`.

Checks. Every cycle succeeds, and `online` equals `possible`.

Expected output. On the 700. The 510 reports `online 0-5`.

```
PASS  SMP cell: 5 create/start/destroy cycles, CPUs back with Linux (online 0-7)
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 247-252; board log 187.

R-20, R-21 PSCI `CPU_ON` allowed and denied

Purpose. A cell starts its other CPUs with the PSCI call `CPU_ON`, which the hypervisor handles.

- **R-20:** the call must start a CPU that belongs to the cell.
- **R-21:** the call must be refused for a CPU outside the cell. Otherwise a cell could take a core from Linux or from another cell.

Procedure. `run_zephyr` in `-zephyr-smp`, 2 s, with a bare-metal test inmate, `psci-probe.S`. Its first CPU prints `A`, calls `CPU_ON` and prints `+` and the result; the started CPU prints `B`.

Test	Inmate	Target CPU
R-20	`psci-probe.bin`	the cell's second CPU
R-21	`psci-probe-deny.bin`	a CPU outside the cell

Checks. The UART1 output since the start, with line ends removed:

- R-20 matches `A\+(0B|B0)`: success, and the second CPU printed. The started CPU and the first CPU's result race for UART1, so either order passes.
- R-21 matches `A\+-`: the call was refused.

Expected output.

```
PASS  SMP cell: PSCI CPU_ON from the cell (psci-probe) (expect A\+(0B|B0))
PASS  SMP cell: PSCI CPU_ON from the cell (psci-probe-deny) (expect A\+-)
```

`psci-probe.log` on the 700:

```
A+0B
```

R-21's output is shorter than four characters, so it is not in `psci-probe-deny.log`. It is in `uart1.log`. Since 2026-10-26 the script also keeps the unfiltered output, as `psci-probe-deny.raw`; no run has produced that file yet.

```
A+-
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 254-261; `psci-probe.S`; board log 187 (`psci-probe.log`, `uart1.log`).

R-22 SMP synchronisation, ten starts

Purpose. An SMP Zephyr image must start both CPUs and run a thread on each, every time. About half the starts once hung at start-up, while `GICR_ISPENDR0` did not show the cell's pending SGIs. This test guards the fix.

Procedure. `run_zephyr` in `-zephyr-smp`, 4 s, ten times, with the SMP build of `samples/synchronization` (`CONFIG_SCHED_CPU_MASK=y`, a thread pinned to each CPU).

Checks. Every run succeeds, and every log `sync-smp-<n>.log` contains `from cpu 0` and `from cpu 1`.

Expected output.

```
PASS  SMP cell: zephyr synchronization, a thread on each CPU, 10 starts
```

`sync-smp-1.log` on the 700:

```
*** Booting Zephyr OS build 2e9e70f0a16c ***
Secondary CPU core 1 (MPID:0x300) is up
thread_a: Hello World from cpu 0 on mt8390_genio_700_evk!
thread_b: Hello World from cpu 1 on mt8390_genio_700_evk!
[...]
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 263-271; board log 187 (`sync-smp-1.log`) to `sync-smp-10.log`.

R-23 Pending SGIs in the redistributor (`sgi-probe`)

Purpose. The pending state of a cell's software-generated interrupts must be correct in `GICR_ISPENDR0` and `GICR_ICPENDR0`. This must also hold when more SGIs are pending than the GIC has list registers, so that some wait in the hypervisor's queue. This is the state behind R-22's start-up hang.

Procedure. `run_zephyr` in `-zephyr`, 2 s, with the bare-metal inmate `sgi-probe.S`. It prints:

- `S`: `ISPENDR0` after sending SGIs 3 to 8 to itself. The GIC has four list registers, so two stay queued.
- `C`: `ISPENDR0` after clearing SGIs 3 and 8 through `ICPENDR0`.
- `P`: `ISPENDR0` after setting SGI 9 through `ISPENDR0`.
- `A`: the interrupts acknowledged afterwards.

Checks. The output matches `S....01f8C....00f0P....02f0A`: SGIs 3-8, then 4-7, then 4-7 and 9 pending. The upper 16 bits (PPIs) are not compared. The acknowledged interrupts, sorted, are exactly `00000004 00000005 00000006 00000007 00000009`.

Expected output.

```
PASS  zephyr cell: GICR_ISPENDR0/ICPENDR0 with queued SGIs (sgi-probe) (acknowledged 00000004
00000005 00000006 00000007 00000009 )
```

`sgi-probe.log` on the 700:

```
S813f01f8C813f00f0P813f02f0A 00000009 00000004 00000005 00000007 00000006
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 273-278; `sgi-probe.S`; board log 187 (`sgi-probe.log`).

R-24 GPIO loopback in the SMP cell

Purpose. As R-08, with the cell on two cores.

Procedure. `run_zephyr` with `gpio/loopback` in `-zephyr-smp`, 12 s.

Checks. `loopback-smp.log` contains `GPIOTEST DONE passed=25 failed=0 -> PASS`.

Expected output.

```
PASS  SMP cell: zephyr sample gpio/loopback
```

The log reads as in R-08.

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 280-282; board log 187 (`loopback-smp.log`).

2.6 Audio in the AFE cells

The AFE cells own the audio front end (AFE) and its eTDM pins. The script prepares the board once:

1. It sets the 16 eTDM pins (4, 5, 6, 11, 107-110, 114-117, 125-128) to mode 0 in the GPIO mode registers. The cell muxes them itself.
2. It keeps the AFE powered: `power/control` of `10b10000.afe` is set to `on`.

Each sample then runs with `run_zephyr` for 10 s and is checked by its type:

Type	Pass criterion, coded in `afe_sample`
`api_reject`	The log contains `PASS -- 15 of 15`
`twostream_*`	The log contains `PASS -- the timing domain is reference counted`
`loopback_*`	The last five `lag` values are identical, and the last `verified` frame count is above 100000
`playback_*`	More than three `playing... DMA offset` lines
`record_*`	More than three `ch0:` lines

In every case, the run must also succeed (Section 2.1). The detail of the `PASS` line is the last `lag` value.

R-25 to R-34 Audio samples in `-zephyr-afe`

Purpose. A Zephyr cell must drive the audio front end on its own while Linux runs. That means:

- argument checking of the driver API;
- two playback streams on one timing domain, stopped and restarted independently;
- loopbacks through the eTDM wires, with a constant lag;
- playback and capture on every memory interface the samples use.

A failure would show a resource that the cell does not fully own: a clock, reset, interrupt or memory interface.

Procedure. `run_zephyr` in `-zephyr-afe`, 10 s per sample, in this order:

Test	Sample	Stream
R-25	`api_reject`	15 invalid calls of `configure()` and `set_buf()`
R-26	`twostream_dl11_dl8`	DL11 and DL8, 16 ch, 48 kHz, stopped and restarted in both orders
R-27	`loopback_dl11_ul8`	DL11 to UL8, I2SO1 wired to TDMIN
R-28	`loopback_dl8_ul3`	DL8 to UL3, I2SO2 wired to I2SIN
R-29	`loopback_dl11_ul9`	DL11 to UL9, 32 ch
R-30	`playback_dl11`	DL11, 32 ch, to eTDM_OUT1 and eTDM_OUT2
R-31	`playback_dl8`	DL8
R-32	`record_ul8`	UL8, 16 ch, from eTDM_IN1
R-33	`record_ul3`	UL3
R-34	`record_ul9`	UL9, 32 ch

Checks. By type, as in the table of Section 2.6.

Expected output. The script's lines on the 700:

```
PASS  zephyr AFE sample audio/api_reject
PASS  zephyr AFE sample audio/twostream_dl11_dl8
PASS  zephyr AFE sample audio/loopback_dl11_ul8 (lag -2)
PASS  zephyr AFE sample audio/loopback_dl8_ul3 (lag 31)
PASS  zephyr AFE sample audio/loopback_dl11_ul9 (lag -10)
PASS  zephyr AFE sample audio/playback_dl11
PASS  zephyr AFE sample audio/playback_dl8
PASS  zephyr AFE sample audio/record_ul8
PASS  zephyr AFE sample audio/record_ul3
PASS  zephyr AFE sample audio/record_ul9
```

`api_reject.log`, abridged:

```
MT8188 AFE argument rejection test
  buffer 0x61000000, 30720 bytes
  configure()
    valid, as a control               ok    got 0        wanted 0
    memory interface out of range     ok    got -EINVAL  wanted -EINVAL
[...]
    base plus size past 32 bits       ok    got -EINVAL  wanted -EINVAL
PASS -- 15 of 15 checks
```

`twostream_dl11_dl8.log`, abridged:

```
  2. DL8 stopped -- DL11 must be untouched
    DL11 survivor          ok     2403 frames in 50 ms, wanted ~2400
    DL8 stopped            ok        0 frames in 50 ms, wanted 0
[...]
PASS -- the timing domain is reference counted
```

`loopback_dl11_ul8.log`, the last lines:

```
[   7s] played 345120 fr  verified 340320 fr  ticks 1419  rot 0 lag -2  isr max 1540 us
[   8s] played 393600 fr  verified 388800 fr  ticks 1621  rot 0 lag -2  isr max 1540 us
[   9s] played 442080 fr  verified 437280 fr  ticks 1822  rot 0 lag -2  isr max 1540 us
```

`playback_dl11.log` and `record_ul8.log`, the first lines; the record line is cut:

```
MT8188 DL11 playback started: 48kHz 32ch 32-bit DSP-B
  ch0-15  -> eTDM_OUT1 (O072-O087)
  ch16-31 -> eTDM_OUT2 (O048-O063), via DL8_DL11 mux
playing... DMA offset 0x133f0 / 0x96000
playing... DMA offset 0x180c0 / 0x96000
```

```
MT8188 UL8 capture started: 48kHz 16ch 32-bit from eTDM_IN1
UL8 ch0: 0x00000000  UL8 ch1: 0x00000000  UL8 ch2: 0x00000000  [...]
```

Last run. Pass on both boards, 2026-10-26. The lags are the same on both boards: −2, 31 and −10 frames (board logs 179 and 187).

Sources. `regression.sh` lines 284-309; board log 187 (`<sample>.log`).

R-35 to R-40 Audio in the AFE cells on the A78 and on two A55 cores

Purpose. The AFE cells also come as `-zephyr-afe-a78`, on a Cortex-A78, and as `-zephyr-afe-smp`, on two Cortex-A55 cores, with single-core images. Audio must work in both, with the AFE's interrupt routed to a big core or to the lowest CPU of a two-core cell.

Procedure. `afe_sample` as in R-25 to R-34, 10 s per sample:

Test	Cell	Sample
R-35	`-zephyr-afe-a78`	`api_reject`
R-36	`-zephyr-afe-a78`	`loopback_dl11_ul8`
R-37	`-zephyr-afe-a78`	`loopback_dl11_ul9`
R-38	`-zephyr-afe-smp`	`api_reject`
R-39	`-zephyr-afe-smp`	`loopback_dl11_ul8`
R-40	`-zephyr-afe-smp`	`loopback_dl11_ul9`

The logs are `<sample>-afe-a78.log` and `<sample>-afe-smp.log`.

Checks. By type, as in Section 2.6.

Expected output. On the 700:

```
PASS  zephyr AFE-a78 cell: audio/api_reject
PASS  zephyr AFE-a78 cell: audio/loopback_dl11_ul8 (lag -2)
PASS  zephyr AFE-a78 cell: audio/loopback_dl11_ul9 (lag -11)
PASS  zephyr AFE-smp cell: audio/api_reject
PASS  zephyr AFE-smp cell: audio/loopback_dl11_ul8 (lag -1)
PASS  zephyr AFE-smp cell: audio/loopback_dl11_ul9 (lag -10)
```

`loopback_dl11_ul9-afe-a78.log`, the last lines:

```
[   7s] played 345120 fr  verified 340320 fr  ticks 1419  lag -11  isr max 270 us
[   8s] played 393600 fr  verified 388800 fr  ticks 1621  lag -11  isr max 270 us
[   9s] played 442080 fr  verified 437280 fr  ticks 1822  lag -11  isr max 270 us
```

Last run. Pass on both boards, 2026-10-26. The lags differ by one frame between runs: on the 510, −1, −11, −1 and −11; on the 700, −2, −11, −1 and −10 (board logs 179 and 187).

Sources. `regression.sh` lines 311-317; board log 187 (`*-afe-a78.log`, `*-afe-smp.log`).

2.7 The root cell's access to shared registers

These checks pin down the boundary between the root cell (Linux) and the cells for registers that both use. Two kinds of register are shared:

- **The pin configuration registers (IOCFG)** are not mediated: by design decision c1-D, Linux keeps direct access to them.
- **The GPIO, EINT, clock and watchdog registers** are mediated by the MediaTek units. A write from Linux applies only to Linux's own pins and bits, and sub-word accesses follow rules of their own.

R-41 IOCFG not mediated: Linux's write reaches a cell's pin

Purpose. The test documents the boundary chosen in c1-D. Pin configuration settings, such as the pull, are not mediated, so Linux can still change them for a pin that a cell owns. If this changed without a decision, by mediation added or by access lost, the test would show it.

Procedure.

1. Save the three IOCFG registers of GPIO 38 (`0x11c000e0`, `0x11c000b0`, `0x11c00010`) and pin 38's line of `pinconf-pins`.
2. Create and start `-zephyr` with `hello_world`. The cell owns GPIO 38.
3. From Linux, write bit 18 of `0x11c000e0` to 1, bit 18 of `0x11c000b0` to 0, and bits 18-20 of `0x11c00010` to 7.
4. Destroy the cell, then restore the three registers.

Checks. Pin 38's line of `pinconf-pins`, read after the cell is destroyed, differs from the line read before. Linux's writes took effect.

Expected output.

```
PASS  IOCFG (not mediated, c1-D): Linux's writes reach a Zephyr pin
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 319-333; board log 187.

R-42 Sub-word reads of a register without entries

Purpose. Linux reads some registers in bytes. For a register that no mediation entry covers, the hypervisor must pass sub-word reads through, consistently with a 32-bit read.

Procedure. Read GPIO `SEC_EN_0` (`0x10005a00`, a register without entries) once as 32 bits and once as four bytes, at offsets 0 to 3.

Checks.

- The four bytes, put together little-endian, equal the 32-bit value.
- `jailhouse cell list` still shows the root cell running on all its CPUs (`running 0-`).

Expected output.

```
PASS  sub-word reads (0 = 0)
```

Last run. Pass on both boards, 2026-10-26: the register reads 0 both ways (board logs 179 and 187).

Sources. `regression.sh` lines 335-347; board log 187.

R-43 Root write carrying a cell's bit (EINT `D0EN_1`, fix J4)

Purpose. A register like EINT `D0EN_1` holds bits of several owners. When Linux writes it, the hypervisor must apply Linux's bits and keep the cell's. Otherwise Linux could switch a cell's interrupt on or off simply by writing back a stale value. Fix J4 of release 1.1 implements this.

Procedure.

1. `D0EN_1` (`0x1000b404`) holds EINTs 32-63. The `-zephyr` cell owns EINT 38, bit 6.
2. Pick bit X: the highest EINT up to 63 that Linux does not use (from `/proc/interrupts`) and that is not 38 or 40.
3. Clear bit 6 and read the register (v1).
4. Create `-zephyr`, without starting it.
5. From Linux, write v1 with bit X flipped and bit 6 set.
6. Destroy the cell, read the register (v2), and restore the original value.

Checks. v1 XOR v2 is exactly bit X. Linux's own bit changed, and the cell's bit 6 stayed clear.

Expected output. Both boards:

```
PASS  root: a write carrying a cell's bit applies its own bits (EINT D0EN_1) (0xffffffbf ->
0x7fffffbf, EINT 63)
```

Last run. Pass on both boards, 2026-10-26, with EINT 63 (board logs 179 and 187).

Sources. `regression.sh` lines 349-366; board log 187.

R-44 Root byte accesses to a mediated register (fix J5)

Purpose. A byte read or write by Linux to a mediated register must not stop the CPU that issued it. The hypervisor ignores the access, reads return 0, and it reports the access once on its console. Fix J5 of release 1.1 implements this. Before it, such an access stopped the root cell.

Procedure. On CPU 1 (`taskset -c 1`), a byte read and a byte write of 0 to GPIO `DIR_0` (`0x10005000`, mediated). Wait 1 s.

Checks.

- The byte read returns 0.
- The root cell's line in `jailhouse cell list` has four fields, so no CPU is listed as failed.
- `hv-run.log` contains exactly one `GPIO: root cell's 1-byte access to 0x10005000 ignored`.

Expected output.

```
PASS  root: byte accesses to a mediated register ignored, reported once, CPUs running (read 0)
```

`hv-run.log`:

```
GPIO: root cell's 1-byte access to 0x10005000 ignored
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 368-376; board log 187 (`hv-run.log`).

2.8 Cell configuration checks and the CPUs

The regression's test cells are `test-mapped`, `test-ok`, `test-busy` and `test-a78`.

R-45 Refusing a cell that maps a mediated window

Purpose. A cell configuration that maps a register window the hypervisor mediates must be refused. Otherwise the cell would bypass the mediation and reach other owners' pins and bits directly.

Procedure. `jailhouse cell create /root/test-mapped.cell`.

Checks. The command's output contains `Invalid argument`.

Expected output.

```
PASS  refuse a cell mapping a mediated window
```

The command's error message is not captured in the logs, and the hypervisor console has no line for the refusal. Since 2026-10-26 the script prints the message as the detail of the `PASS` line; no run has produced it yet.

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 378-381; the test cell `test-mapped`.

R-46 Refusing a cell that claims another cell's pin

Purpose. Two cells must not own the same pin. The second must be refused while the first exists.

Procedure. `jailhouse cell create /root/test-ok.cell`, then `jailhouse cell create /root/test-busy.cell`, which claims a pin of `test-ok`.

Checks. The first create succeeds, and the second one's output contains `busy`.

Expected output.

```
PASS  refuse a cell claiming another cell's pin
```

The second command's error message is not captured in the logs. Since 2026-10-26 the script prints it as the detail of the `PASS` line; no run has produced it yet. `hv-run.log` shows the first cell only:

```
Created cell "test-ok"
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 382-384; board log 187 (`hv-run.log`).

R-47 Pin returned to Linux after destroy

Purpose. When a cell is destroyed, its pins must become free again. Otherwise a later cell cannot claim them.

Procedure. Destroy `test-ok`, then create and destroy `test-busy`.

Checks. The create and the destroy succeed.

Expected output.

```
PASS  pin returned to Linux after destroy
```

`hv-run.log`:

```
Closing cell "test-ok"
[...]
Created cell "test-busy"
[...]
Closing cell "test-busy"
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 385-388; board log 187 (`hv-run.log`).

R-48 Big cores: cell cycles and Linux hotplug

Purpose. A big core that a cell has used must still take part in Linux's CPU hotplug afterwards.

Procedure.

1. Create and destroy `test-a78` three times. It uses CPU 5 on the 510 and CPU 7 on the 700.
2. Take each big core offline and online again from Linux: CPUs 4 and 5 on the 510, 6 and 7 on the 700.

Checks. Every create and destroy succeeds, and `online` equals `possible`.

Expected output. On the 700. The 510 reports `online 0-5`.

```
PASS  big cores: cell cycles and Linux hotplug (online 0-7)
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 390-397; board log 187.

2.9 End of the run: console, disable and re-enable

R-49 Root write to a pin configuration register not trapped

Purpose. This is the companion to R-41. Linux's writes to IOCFG registers must reach the hardware without a trap, and the hypervisor must not log them as ignored.

Procedure.

1. Read `0x11c00004` and write the same value back. The register reads 0, so the write changes nothing either way.
2. Wait 1 s, stop `hv-console`, and copy `/root/hv-run.log` to the run's directory.

Checks. `hv-run.log` does not mention `0x11c00004`.

Expected output.

```
PASS  root (not mediated, c1-D): write to a pin configuration register not trapped
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 399-406; board log 187 (`hv-run.log`).

R-50 The hypervisor console of the run

Purpose. The hypervisor console must show no fault during the run. The checks above test outcomes. This check catches a problem that does not change an outcome: an unhandled access, a crash, a refused firmware call, or an ignored root write.

Procedure. Read `hv-run.log`, the console from R-01 to R-49.

Checks.

- The log contains `Created cell`, so it covers the run.
- No line matches `Unhandled`, `FATAL`, `Denied SiP` or `undescribed registers ignored` (case-insensitive), except lines that name `(0x11c00004)`.

Expected output.

```
PASS  hypervisor console of the run: no unhandled access, crash, denied SiP call or ignored root
write
```

On the 700, `hv-run.log` (434 lines) contains:

- the enable messages, as in R-01;
- 70 `Created cell` and 70 `Closing cell` lines, and 64 each of `Cell ... can be loaded` and `Started cell`;
- page-pool lines, `mem 96/988` to `mem 113/988`;
- R-44's line.

It has no other message.

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 408-411; board log 187 (`hv-run.log`).

R-51 Disable and re-enable

Purpose. The hypervisor must hand all CPUs and devices back to Linux on `jailhouse disable`, and enable again in the same boot. Products update the cell set this way without a reboot.

Procedure. Destroy every non-root cell, `jailhouse disable`, then `jailhouse enable genio-<board>-evk.cell`.

Checks. Both commands succeed, and `jailhouse cell list` shows a cell `running`.

Expected output.

```
PASS  disable and re-enable
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 413-417; board log 187.

R-52 The hypervisor console after the re-enable

Purpose. The second enable must be as clean as the first.

Procedure. Save `jailhouse console` as `hv-end.log`.

Checks. No line matches `Unhandled` or `FATAL` (case-insensitive).

Expected output.

```
PASS  hypervisor console after re-enable: no unhandled access or crash
```

`hv-end.log` on the 700 ends as the first enable did:

```
Initializing unit: mt8188_wdt
[...]
Adding virtual PCI device 00:00.0 to cell "genio-700-evk"
Page pool usage after late setup: mem 96/988, remap 532/131072
Activating hypervisor
```

The script ends with the summary:

```
=== 52 passed, 0 failed
```

Last run. Pass on both boards, 2026-10-26 (board logs 179 and 187).

Sources. `regression.sh` lines 419-425; board log 187 (`hv-end.log`).

03

The watchdog set W1

3.1 Overview

3.2 Stage M2: the watchdog unit

3.3 Stage M3: reset causes, PSCI resets and faults

3.4 Stage Z: Zephyr watchdog applications

3.5 Stage TWO: two cells with watchdogs

3.6 Stage D: the restart daemon

3.1 Overview

What W1 provides. Release 1.1 gives every cell that has a watchdog entry in its configuration a virtual watchdog, at the address of the SoC's TOPRGU (`0x10007000`):

- **Arming and kicking.** A cell that arms the watchdog (`MODE`, `LENGTH`) and stops kicking it (`RESTART`) fails with "watchdog timeout". Only the cell fails; Linux and the other cells go on.
- **Resets.** A software reset through `SWRST` fails the cell with "chip reset requested". The PSCI calls `SYSTEM_RESET` and `SYSTEM_RESET2` fail it with "system reset requested".
- **Reset cause.** The next cell of the same name reads the cause in its `STA`: bit 31 after a timeout, bit 30 after a software or system reset.
- **Subsystem resets.** `SWSYSRST` resets only the subsystems that the cell owns. The audio front end is one, in the AFE cells.
- **Deadline checks.** The hypervisor checks the cells' deadlines on ticks from the root cell's driver (module parameter `tick_ms`, 100 ms).
- **Linux's watchdog.** Linux keeps its own watchdog (the hardware TOPRGU, 31 s, fed by systemd), through the watchdog unit.
- **Restarts.** The daemon `jailhouse-cell-restartd` restarts failed cells, as `/etc/jailhouse/cell-restart.conf` says.

The script. `rity-w1.sh <board> <log prefix> <stages>` runs the W1 set on an image that ships Jailhouse. It uses the image's hypervisor, module, cells, daemon and unit as they are. The stages are M2 (Section 3.2), M3 (3.3), Z (3.4), TWO (3.5) and D (3.6). It brings together the milestone scripts `m2-test.sh`, `m3-test.sh`, `m3-two.sh` and `m3-daemon.sh`, and stage Z repeats the runs of `wdt-sysreset.sh` and `hwinfo-v2.sh`.

Before the stages, the script:

1. Stops if it does not see exactly one adb device.
2. Prints the md5 of the image's hypervisor, module and daemon.
3. Stops the image's daemon; the end of the script restores it.
4. Pushes the test inmates, the Zephyr watchdog applications of the rel-1.1 set, the test cell `wdt-b` and the rpmsg sample to `/root`.
5. Enables Jailhouse if needed, and follows the hypervisor console into `/root/hv-w1.log`, which it saves as `<prefix>-hv.log`.
6. Runs a Zephyr `hello_world`, so that UART1 is set up for the bare-metal probes (Section 1.3).

How a case runs. Most cases use the script's function `run`:

1. Capture UART1 into `<prefix>-<case>-uart1.log`.

2. Create the cell, load the image at `0x8000` and start it, while `poll-state.py` waits on the cell's `state` file in sysfs.
3. Print when `poll()` woke and the state then (`poll: woke after ... s`), the cell list, the end of the UART1 output, and the last three console lines about the watchdog or a failure.

The probe. Most cases run the bare-metal test inmate `wdt-probe.S`. It is built in eleven variants. Every variant first prints its number and `s<STA>`, the cell's `STA` in hex, unless it is built for a cell without a watchdog (`NOSTA`).

Variant	What it does, and what it prints after `<n>s<STA>`
1	`LENGTH` 2 s, `MODE.EN`; prints `m<MODE>` and `l<LENGTH>` as read back; kicks five times about 1 s apart (`K`), then stops (`S`)
2	Writes `SWRST` with a wrong key (`w`), waits 1 s, then writes the key (`R`)
3	Asserts subsystem resets 14 (audio) and 16 in `SWSYSRST`, reads back (`a<value>`), releases them (`r<value>`), then `D`
4	`LENGTH` 3 s, `MODE.EN` (`A`), and never kicks
5	PSCI `SYSTEM_RESET`. If the call returns: `F` and the result
6	Writes the lock bit of the audio reset to `SWSYSRST_LOCK` (`k<readback>`), reads `WDT_INTERNAL` (`i<value>`), writes all ones to `STA` (`t<readback>`); then stays idle
7	`CPU_ON` for the cell's second CPU, which prints `b` and spins; `+<result>`; then arms 3 s (`A`) and spins with interrupts masked
8	Vectors that spin; arms 3 s (`U`), then an undefined instruction
9	`X`, then a read of `0x10000000`, which no cell maps
10	`LENGTH` 2 s, `MODE.EN`, a kick about every second (`k`), forever
11	`PSCI_FEATURES` of `SYSTEM_RESET2` (`f`) and `SYSTEM_RESET` (`g`), `SYSTEM_RESET2` of type 1 (`e`), then the warm reset, which must not return (else `F`). Prints `R` for its number

The verdict. The W1 scripts have no coded verdict. They print observations: what the probe prints, the cell's state, when `poll()` woke, the console lines, and the daemon's journal. A run is judged against the criteria of the W1 milestone notes. The Checks field of each entry gives those criteria.

Last run. All W1 entries ran on 2026-10-22 on the heads image: the 700 from 11:40 to 11:54 (board log 157), and the 510 from 12:49 to 13:03 (board log 166). The expected output below is from the 700. The 510 printed the same characters and states, with timings within 0.1 s of the 700's. The console reported deadlines 2 to 97 ms late, apart from the tick test W1-M2-T06, and the kernel logged 0 warnings on either board.

Two entries ran again on 2026-10-26 on the 700, refinement-1 image, after script fixes: W1-TWO (board log 188), whose case 2 the 2026-10-22 runs did not exercise, and W1-M3-T18 in its new form (board log 189).

3.2 Stage M2: the watchdog unit

W1-M2-T01 Linux's watchdog through the unit

Purpose. Linux's own watchdog must keep working under the hypervisor. The watchdog unit takes over the TOPRGU registers. If Linux's settings or kicks did not reach the hardware, the SoC would reset after 31 s, or the safety net would be lost without notice.

Procedure. From Linux, read `MODE` (`0x10007000`) and `LENGTH` (`0x10007004`). Wait 40 s, print the uptime before and after, and count the console lines `WDT: no tick`.

Checks (W1 milestone notes).

- `MODE` reads `0x5` and `LENGTH` reads `0xf800`, as Linux set them, read through the unit.
- The SoC is still up after 40 s, more than the 31 s timeout.
- There is no self-check report: the count of `WDT: no tick` lines is 0.

Expected output.

```
=== T1 root: MODE as Linux left it, and Linux's kicks for 40 s (> 31 s)
0x10007000: 0x5
0x10007004: 0xf800
alive, uptime 486.16 -> 526.16
0
```

Last run. As the criteria on both boards, 2026-10-22 (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T02 Kick, then stop

Purpose. A cell that arms its watchdog and then stops kicking must fail, about one timeout after its last kick, and only then. This is the basic function that products rely on to catch a hung cell.

Procedure. `run` with probe 1 in `-zephyr`, 15 s window.

Checks (W1 milestone notes).

- The probe reads back `MODE` 1 and `LENGTH` `0x1000`.
- It kicks five times, then stops.
- The cell fails about 2 s after the last kick, about 7 s after its start, with "missed its deadline" and "watchdog timeout".

Expected output.

```
poll: woke after 7.37 s, events [(3, 10)], state failed
0       genio-700-evk           running           0-2,4-7
1       zephyr                  failed            3
UART1: 1s00000000m00000001l00001000KKKKKS
WDT: cell "zephyr" missed its deadline, detected 92 ms late
Cell "zephyr" failed: watchdog timeout
```

Last run. As the criteria: failed 7.30 s (510) and 7.37 s (700) after the start (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; `wdt-probe.S`; board log 157.

W1-M2-T03 Never kick, in the SMP cell

Purpose. The timeout must stop a cell with two CPUs as a whole, and give both CPUs back to Linux when the cell is destroyed.

Procedure. `run` with probe 4 in `-zephyr-smp`, 10 s window, then destroy.

Checks (W1 milestone notes). The cell fails about 3 s after its start, with both CPUs stopped. After the destroy, all CPUs are online.

Expected output.

```
poll: woke after 3.35 s, events [(3, 10)], state failed
1       zephyr-smp              failed            2-3
UART1: 4s00000000A
WDT: cell "zephyr-smp" missed its deadline, detected 74 ms late
Cell "zephyr-smp" failed: watchdog timeout
destroy rc=0
online 0-7
```

Last run. As the criteria: 3.29 s (510) and 3.35 s (700) (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T04 Software reset fails only the cell

Purpose. On the real TOPRGU, a write of the key to `SWRST` resets the whole SoC. From a cell, it must fail only that cell. A write without the key must change nothing, as on the hardware.

Procedure. `run` with probe 2 in `-zephyr`, 10 s window. Then print the SoC's uptime.

Checks (W1 milestone notes).

- The write with the wrong key is ignored: the cell goes on, the probe prints `R` 1 s later.
- The write with the key fails the cell about 1 s after its start, with "chip reset requested".
- The SoC's uptime runs on.

Expected output. `s80000000` is bit 31, because the previous `zephyr` cell (W1-M2-T02) timed out.

```
poll: woke after 1.30 s, events [(3, 10)], state failed
1       zephyr                  failed            3
UART1: 2s80000000wR
Cell "zephyr" failed: chip reset requested
SoC uptime 570.62
```

Last run. As the criteria: 1.33 s (510) and 1.30 s (700) (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T05 Subsystem reset ownership

Purpose. `SWSYSRST` resets SoC subsystems. A cell may reset only those it owns: the AFE cells own the audio reset (bit 14), the other cells none. A reset of a subsystem in use by Linux would crash Linux's driver.

Procedure. `run` with probe 3, 3 s window, first in `-zephyr-afe`, then in `-zephyr`. The probe asserts bits 14 and 16, reads back, releases them and reads back again.

Checks (W1 milestone notes).

- In `-zephyr-afe`, the read-back is `0x4000` while asserted and 0 after release. The console reports `may not reset subsystems 10000, ignored` once.
- In `-zephyr`, the read-backs are 0 and 0, and the console reports `14000`.
- Both cells keep running.

Expected output.

```
UART1: 3s00000000a00004000r00000000D
WDT: cell "zephyr-afe" may not reset subsystems 10000, ignored
[...]
UART1: 3s40000000a00000000r00000000D
WDT: cell "zephyr" may not reset subsystems 14000, ignored
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T06 Ticks from Linux stopped and resumed

Purpose. The hypervisor checks deadlines on Linux's ticks. If the ticks stop, it must not fail cells by mistake, and it must not stay silent either. It reports that it no longer checks, then catches up when the ticks resume.

Procedure.

1. Start `-zephyr` with probe 4, armed for 3 s.
2. Set `tick_ms` to 0, wait 20 s, and read the cell's state.
3. Set `tick_ms` to 100, wait 1 s, and read the state again.

Checks (W1 milestone notes).

- The cell is still running after 20 s without ticks.
- The console reports "no tick from the root cell for ... ms" once.
- After the resume, it reports "ticks from the root cell resumed", and the cell fails at once, about 17 s late.

Expected output.

```
after 20 s without ticks: running
1 s after resuming: failed
WDT: no tick from the root cell for 6994 ms, the cells' watchdogs are not checked
WDT: ticks from the root cell resumed
WDT: cell "zephyr" missed its deadline, detected 17013 ms late
Cell "zephyr" failed: watchdog timeout
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T07 A reload disarms the watchdog

Purpose. Loading a new image into a cell must disarm the old image's watchdog. Otherwise the new image would fail on the old one's deadline.

Procedure. Start `-zephyr` with probe 4, armed for 3 s. After 1 s, load `hello_world` into the same cell and start it. Read the state 6 s later.

Checks (W1 milestone notes). The cell is still running 6 s later.

Expected output.

```
start rc=0
start rc=0
6 s later: running
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T08, W1-M2-T09 Destroy and disable with the watchdog armed

Purpose. An armed watchdog must not get in the way of destroying its cell (T08) or disabling the hypervisor (T09). A deadline must not fire afterwards.

Procedure.

1. Start `-zephyr` with probe 4 and destroy it after 1 s.
2. Start it again. After 1 s, stop the console follower, then `jailhouse disable` and `jailhouse enable`, and list the cells.

Checks (W1 milestone notes). The destroy, the disable and the enable all succeed, and only the root cell is listed, with all CPUs.

Expected output.

```
start rc=0
destroy rc=0
start rc=0
disable rc=0
enable rc=0
0       genio-700-evk           running           0-7
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

W1-M2-T10 Linux's subsystem resets and a cell's reset bit

Purpose. Linux keeps the right to reset the subsystems it owns through `SWSYSRST`. But while an AFE cell exists, Linux's writes must not touch the audio reset bit, which the cell owns.

Procedure. From Linux, on `SWSYSRST` (`0x10007018`):

1. Write `0x88010000` (the key and bit 16), then `0x88000000`. Read back after each.
2. Create `-zephyr-afe`. Write `0x88004000` (bit 14) and read back. Write `0x88000000`.
3. Destroy the cell and read back.

Checks (W1 milestone notes).

- Linux's own bit 16 is set (`0x10000`), then cleared.
- While the AFE cell exists, Linux's write of bit 14 leaves it 0.

Expected output.

```
0x10007018: 0x10000
0x10007018: 0
0x10007018: 0
0x10007018: 0
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M2; board log 157.

3.3 Stage M3: reset causes, PSCI resets and faults

W1-M3-T11 Reset cause after a timeout, and register isolation

Purpose.

- **Reset cause.** A restarted application must be able to tell why it restarted. After a timeout, the next cell of the same name reads bit 31 in `STA`. This holds whether it was reloaded in place or destroyed and created again, and a healthy destroy clears it.
- **Isolation.** The registers of the virtual watchdog must not reach the hardware: the cell's `SWSYSRST_LOCK` and `WDT_INTERNAL`, and writes to `STA`.

Procedure.

1. From Linux, read the hardware's `SWSYSRST_LOCK`, `WDT_INTERNAL` and `STA`.
2. Probe 4 in `-zephyr` times out. Then load probe 6 into the failed cell and start it in place. Read the lock and `WDT_INTERNAL` from Linux again.
3. Destroy the cell, create it with probe 6 (a healthy destroy).
4. Probe 4 times out again. Destroy, create with probe 6 (kept by name).

Checks (W1 milestone notes).

- In place, after the timeout, `s80000000`.
- After a healthy destroy and create, `s00000000`.
- After a second timeout, a destroy and a create, `s80000000` again.
- A write of all ones to `STA` reads back unchanged.
- The cell's write of the lock bit reads back 0 in the cell, and Linux still reads 0 from the hardware.
- `WDT_INTERNAL` reads 0 in the cell, while Linux reads `0xfff` before and after.

Expected output.

```
root: LOCK 0x10007070: 0 INTERNAL 0x10007010: 0xfff STA 0x1000700c: 0x40000000
=== 11:43:12 T11-timeout: genio-700-evk-zephyr.cell with wdt-probe-4.bin
[...]
UART1: 4s00000000A
Cell "zephyr" failed: watchdog timeout
=== 11:43:25 T11-in-place
UART1: 6s80000000k00000000i00000000t80000000
root: LOCK 0x10007070: 0 INTERNAL 0x10007010: 0xfff
=== 11:43:29 T11-healthy-destroy
UART1: 6s00000000k00000000i00000000t00000000
[...]
=== 11:43:47 T11-by-name
UART1: 6s80000000k00000000i00000000t80000000
```

The hardware `STA` that Linux reads (`0x40000000`) is the SoC's own, from its last reboot. The cells never see it.

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T12 Reset cause after a software reset

Purpose. As W1-M3-T11, for a software reset: bit 30.

Procedure. `run` with probe 2 in `-zephyr`, 10 s window. Destroy, then create with probe 6.

Checks (W1 milestone notes). The cell fails after about 1 s, with "chip reset requested". The next cell reads `s40000000`.

Expected output.

```
poll: woke after 1.29 s, events [(3, 10)], state failed
UART1: 2s00000000wR
Cell "zephyr" failed: chip reset requested
[...]
UART1: 6s40000000k00000000i00000000t40000000
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T13 PSCI `SYSTEM_RESET` fails the cell

Purpose. An operating system reboots through PSCI `SYSTEM_RESET`. From a cell with a watchdog, the call must fail only the cell, never the SoC. Like the reboot on real hardware, it must not return.

Procedure. `run` with probe 5 in `-zephyr`, 10 s window. Destroy, then create with probe 6.

Checks (W1 milestone notes).

- The call does not return: no `F`.
- The cell fails at the first tick after the call, with "system reset requested".
- The next cell reads `s40000000`.

Expected output.

```
poll: woke after 0.28 s, events [(3, 10)], state failed
UART1: 5s00000000
Cell "zephyr" failed: system reset requested
[...]
UART1: 6s40000000k00000000i00000000t40000000
```

Last run. As the criteria: failed 0.38 s (510) and 0.28 s (700) after the start (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T14 `SYSTEM_RESET` in a cell without a watchdog

Purpose. A cell without a watchdog entry has nothing to restart it with. For such a cell, `SYSTEM_RESET` must return `NOT_SUPPORTED`, and the cell must keep running. The caller can then handle the reset itself.

Procedure. Create `-uart-demo`, which has no watchdog entry. Load probe 5, built without the `STA` read (`NOSTA`), at address 0, and start it. List the cells after 2 s.

Checks (W1 milestone notes). The call returns `ffffffff` (`NOT_SUPPORTED`), and the cell keeps running.

Expected output.

```
1       uart-demo               running          3
UART1: 5Fffffffff
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T15 SMP cell, both CPUs busy

Purpose. The watchdog must also catch a cell whose CPUs are all busy with interrupts masked. The hypervisor's check must not depend on the cell taking an interrupt or a trap.

Procedure. `run` with probe 7 in `-zephyr-smp`, 10 s window.

Checks (W1 milestone notes).

- `CPU_ON` returns 0, and the second CPU prints `b` and spins.
- The first CPU arms the watchdog and spins without `WFE`, interrupts masked.
- The cell fails after about 3 s with both CPUs stopped, and Linux has its CPUs back after the destroy.

Expected output. The second CPU's `b` comes before the first CPU's result, `00000000`.

```
poll: woke after 3.37 s, events [(3, 10)], state failed
1       zephyr-smp              failed            2-3
UART1: 7s00000000+b00000000A
WDT: cell "zephyr-smp" missed its deadline, detected 95 ms late
Cell "zephyr-smp" failed: watchdog timeout
destroy rc=0
online 0-7
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T16 A fault inside the cell

Purpose. A cell whose software has crashed into its own fault handler looks alive to the hypervisor. Its watchdog must still fail it.

Procedure. `run` with probe 8 in `-zephyr`, 10 s window.

Checks (W1 milestone notes). After an undefined instruction, with exception vectors that spin, the cell fails after about 3 s.

Expected output.

```
poll: woke after 3.35 s, events [(3, 10)], state failed
UART1: 8s00000000U
WDT: cell "zephyr" missed its deadline, detected 95 ms late
Cell "zephyr" failed: watchdog timeout
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T17 A stage-2 fault fails the cell at once

Purpose. An access outside the cell's configuration is a stage-2 fault. The hypervisor must fail the cell at once, and the driver must notify `poll()`, so that the restart daemon reacts without delay.

Procedure. `run` with probe 9 in `-zephyr`, 10 s window.

Checks (W1 milestone notes).

- The console shows "Unhandled data read at 0x10000000", and the CPU is parked.
- The cell fails, and `poll()` wakes at once.

Expected output. `s80000000` comes from W1-M3-T16's timeout.

```
poll: woke after 0.28 s, events [(3, 10)], state failed
1       zephyr                  failed            3                       3
UART1: 9s80000000X
Unhandled data read at 0x10000000(4)
FATAL: unhandled trap (exception class 0x24)
Parking CPU 3 (Cell: "zephyr")
```

Last run. As the criteria: `poll()` woke after 0.29 s (510) and 0.28 s (700) (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

W1-M3-T18 A second shipped cell on UART1 is refused

Purpose. The shipped Zephyr cells all claim UART1's clock gate and header pins, so only one of them can exist at a time. The hypervisor must refuse a second one with `EBUSY`, rather than give two cells the same UART. Two cells with watchdogs at once are tested by W1-TWO, with a test cell that has no UART.

Procedure. Start `-zephyr-a78` with probe 10, which kicks. Create `-zephyr` and print the return code and the message. Then destroy `-zephyr-a78`.

Checks. Coded, as a line of its own: `T18: EBUSY, as expected` if the create is refused with "busy"; otherwise `T18: created, NOT the expected EBUSY` or `T18: refused, but NOT with EBUSY`. The line does not count towards a summary (Section 7.1).

Expected output. Board log 189, the 700:

```
=== T18 a second cell on UART1 while -zephyr-a78 (kicking) holds it: refused with EBUSY. Two
cells with watchdogs at once: stage TWO
start rc=0
create genio-700-evk-zephyr.cell: rc=1 JAILHOUSE_CELL_CREATE: Device or resource busy
T18: EBUSY, as expected
0       genio-700-evk           running           0-6
1       zephyr-a78              running           7
destroy rc=0
online 0-7
```

Last run. As expected on the 700, 2026-10-26, refinement-1 image, with the stage run on its own (board log 189). In the runs of 2026-10-22 (board logs 157 and 166), the earlier form of the case met the same `EBUSY`, but went on to start the cell that did not exist, and printed a traceback.

Sources. `rity-w1.sh`, stage M3; board log 189.

W1-M3-T19 The cells' watchdogs without Linux's

Purpose. The cells' watchdogs must not depend on Linux's own watchdog being armed. The ticks come from the Jailhouse driver, not from systemd's watchdog feeder.

Procedure.

1. Stop Linux's feeder with a runtime drop-in (`RuntimeWatchdogSec=0` in `/run/systemd/system.conf.d/`) and `systemctl daemon-reexec` .
2. If `MODE.EN` is still set, restore the feeder at once and skip the case.
3. `run` with probe 4 in `-zephyr` . Wait 35 s with no feeder.
4. Remove the drop-in, re-execute systemd, and read `MODE` and `LENGTH` .

Checks (W1 milestone notes).

- `MODE` reads `0x4` : Linux's watchdog is off.
- Probe 4 still fails the cell after about 3 s.
- The SoC runs 35 s with no feeder.
- After the restore, `RuntimeWatchdogUSec=31s` , `MODE 0x5` , `LENGTH 0xf800` .

Expected output.

```
RuntimeWatchdogUSec=0
MODE 0x10007000: 0x4
[...]
poll: woke after 3.26 s, events [(3, 10)], state failed
UART1: 4s00000000A
WDT: cell "zephyr" missed its deadline, detected 4 ms late
Cell "zephyr" failed: watchdog timeout
[...]
no feeder for 35 s: uptime 770.37 -> 805.38, MODE 0x10007000: 0x4
RuntimeWatchdogUSec=31s
MODE 0x10007000: 0x5 LENGTH 0x10007004: 0xf800
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh` , stage M3; board log 157.

W1-M3-T20 Linux's watchdog across disable and enable

Purpose. Disabling and enabling the hypervisor must leave Linux's watchdog as Linux set it, and its kicks must still reach the hardware afterwards.

Procedure. Stop the console follower, then `jailhouse disable` and `jailhouse enable` , reading `MODE` and `LENGTH` after each. Wait 40 s and count the `WDT: no tick` lines.

Checks (W1 milestone notes).

- `MODE` stays `0x5` and `LENGTH 0xf800` .
- The SoC is alive after 40 s of Linux's kicks, with no self-check report.

Expected output.

```
disable rc=0
MODE 0x10007000: 0x5 LENGTH 0x10007004: 0xf800
enable rc=0
MODE 0x10007000: 0x5 LENGTH 0x10007004: 0xf800
alive, uptime 808.09 -> 848.10
0
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage M3; board log 157.

3.4 Stage Z: Zephyr watchdog applications

Stage Z runs the Zephyr watchdog applications of the rel-1.1 set (the set's watchdog applications for each board) in `-zephyr`: the Genio sample `watchdog/expiry`, Zephyr's `task_wdt` sample, and Zephyr's tests `hwinfo_api`, `wdt_basic_api` and `wdt_error_cases`. They check the virtual watchdog through the Zephyr driver that customers use. At the end, the script counts the suite results and reset causes in each UART1 log.

W1-Z-EXPIRY Watchdog expiry sample

Purpose. The Genio sample `watchdog/expiry` shows the documented use of the watchdog: arm, kick, stop kicking. The timeout must fall when the driver says it is due.

Procedure. `run` with the `expiry` image in `-zephyr`, 25 s window. The sample arms 5 s, kicks every second, stops kicking at 10 s, and prints `alive` lines until the cell fails.

Checks (W1 milestone notes). The cell fails about 15.0 s after the arm, when the sample says the watchdog is due.

Expected output.

```
WDTEXPIRY armed for 5000 ms, counter at 13000000 Hz
WDTEXPIRY arm: 5 ms since the arm, cycle 11158227944
WDTEXPIRY kick 1: 1000 ms since the arm, cycle 11171158800
[...]
WDTEXPIRY stopped kicking; the watchdog is due at 15004 ms
[...]
WDTEXPIRY alive: 15004 ms since the arm, cycle 11353210737
```

```
poll: woke after 15.30 s, events [(3, 10)], state failed
WDT: cell "zephyr" missed its deadline, detected 36 ms late
Cell "zephyr" failed: watchdog timeout
```

Last run. As the criteria on both boards, 15.30 s (700) and 15.35 s (510) after the start (board logs 157 and 166).

Sources. `rity-w1.sh`, stage Z; board log 157.

W1-Z-TASKWDT Task watchdog sample and reboot through PSCI

Purpose. Zephyr's `task_wdt` sample detects a stuck thread and reboots the system. In a cell, the reboot must go through PSCI `SYSTEM_RESET` and fail only the cell. It must come before the hardware fallback of the watchdog.

Procedure. `run` with the `task_wdt` image in `-zephyr`, 20 s window. The image has a 2 KiB control-thread stack and PSCI CPU operations.

Checks (W1 milestone notes).

- The UART shows "Main thread still alive...", "Control thread getting stuck...", "Task watchdog channel 1 callback, thread: control" and "Resetting device...".
- The cell fails about 3.6 s after its start, with "system reset requested".

Expected output.

```
Task watchdog sample application.
Main thread still alive...
Control thread started.
[...]
Control thread getting stuck...
Task watchdog channel 1 callback, thread: control
Resetting device...
```

```
poll: woke after 3.83 s, events [(3, 10)], state failed
[...]
Cell "zephyr" failed: system reset requested
```

Last run. As the criteria: 3.83 s (700) and 3.87 s (510) (board logs 157 and 166).

Sources. `rity-w1.sh`, stage Z; board log 157.

W1-Z-HWINFO Reset cause through `hwinfo`

Purpose. An application reads why it restarted through Zephyr's `hwinfo` API, from the virtual `STA`. The API must work after each kind of start: after a watchdog timeout, after a system reset, and after a power-on.

Procedure. `run` with the `hwinfo_api` test image in `-zephyr`, 8 s window, three times:

1. after W1-Z-EXPIRY's timeout;
2. after W1-Z-TASKWDT's system reset;
3. fresh, at the end of stage Z, after a healthy destroy.

Checks (W1 milestone notes, hwinfo v2). Each run passes 3 tests and skips 1 (`test_device_id_get`).

Expected output. The same in all three runs:

```
SUITE PASS - 100.00% [hwinfo_device_id_api]: pass = 3, fail = 0, skip = 1, total = 4 duration =
0.004 seconds
PROJECT EXECUTION SUCCESSFUL
```

Last run. As the criteria on both boards, all three runs (board logs 157 and 166).

Sources. `rity-w1.sh`, stage Z; board log 157.

W1-Z-WDTBASIC Zephyr's basic watchdog API test

Purpose. Zephyr's `wdt_basic_api` test lets the watchdog reset the system on its first boot, and checks the result on its second boot. In a cell, the second boot is the cell's restart.

Procedure.

1. `run` with the `wdt_basic_api` image in `-zephyr`, 10 s window. The cell fails.
2. `run` again, 15 s window: the cell still exists, so the create and the load are refused (`File exists`), and the start restarts the failed cell in place. This is the second boot, as the restart daemon would do it.

Checks (W1 milestone notes). The second boot passes the suite.

Expected output. First boot:

```
Running TESTSUITE wdt_basic_test_suite
[...]
Testcase: test_wdt_no_callback
- pausing watchdog by debugger is not supported
Waiting to restart MCU
```

```
poll: woke after 2.34 s, events [(3, 10)], state failed
[...]
WDT: cell "zephyr" missed its deadline, detected 72 ms late
Cell "zephyr" failed: watchdog timeout
```

Second boot:

```
JAILHOUSE_CELL_CREATE: File exists
load failed
start rc=0
[...]
SUITE PASS - 100.00% [wdt_basic_test_suite]: pass = 1, fail = 0, skip = 0, total = 1 duration =
0.005 seconds
PROJECT EXECUTION SUCCESSFUL
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage Z; board log 157.

W1-Z-WDTERR Zephyr's watchdog error-case test

Purpose. Zephyr's `wdt_error_cases` test checks the driver's answers to invalid and unsupported requests.

Procedure. `run` with the `wdt_error_cases` image in `-zephyr`, 25 s window.

Checks (W1 milestone notes). The suite passes: 16 pass, 7 skip.

Expected output.

```
SUITE PASS - 100.00% [wdt_coverage]: pass = 16, fail = 0, skip = 7, total = 23 duration = 0.049
seconds
PROJECT EXECUTION SUCCESSFUL
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage Z; board log 157.

3.5 Stage TWO: two cells with watchdogs

W1-TWO Two cells with watchdogs at once

Purpose. Two cells must be able to use their watchdogs at the same time. One cell's timeout must not fail the other, and one cell's kicks must not keep the other alive.

Procedure. With the test cell `wdt-b`: CPU 5, 64 KiB at `0x6b400000`, only a watchdog entry, and no UART, pins or clock gates.

1. Start `wdt-b` with probe 10, built without UART output; it kicks every second with a 2 s timeout. Then `-zephyr` with probe 4, which never kicks. Wait 20 s.
2. The other way round: start `-zephyr` with probe 10, which kicks. Then `wdt-b` with probe 4 built without UART output (`wdt-probe-4-nouart.bin`). Wait 20 s.

After each case, the script prints the cell list and the last console lines about the watchdog, a failure or an unhandled access.

Checks (W1 milestone notes, board logs 106 and 188).

- Case 1: `-zephyr` fails after about 3 s, and `wdt-b` is still running 20 s later.
- Case 2: `wdt-b` fails with "watchdog timeout", with no failed CPU and no `Unhandled` line, and `-zephyr`, kicking, keeps running.

Expected output. Board log 188, the 700:

```
=== 13:16:15 T18 wdt-b kicks (probe 10), zephyr does not (probe 4)
wdt-b start rc=0
zephyr start rc=0
poll: woke after 3.26 s, events [(3, 10)], state failed
0       genio-700-evk           running           0-2,4,6-7
1       wdt-b                   running           5
2       zephyr                  failed            3
20 s later:
[...]
1       wdt-b                   running           5
2       zephyr                  failed            3
WDT: cell "zephyr" missed its deadline, detected 11 ms late
Cell "zephyr" failed: watchdog timeout
[...]
=== 13:16:38 and the other way: wdt-b never kicks (probe 4 without UART output: wdt-b has no
UART), zephyr kicks (probe 10, with UART)
zephyr start rc=0
wdt-b start rc=0
0       genio-700-evk           running           0-2,4,6-7
1       zephyr                  running           3
2       wdt-b                   failed            5
[...]
WDT: cell "wdt-b" missed its deadline, detected 47 ms late
Cell "wdt-b" failed: watchdog timeout
```

The case headings still carry the label `T18` of the milestone script.

Last run. As the criteria on the 700, 2026-10-26, refinement-1 image, with the same hypervisor as on 2026-10-22 (board log 188). The hypervisor log has no `Unhandled` line, and the kernel logged 0 warnings. The 510 was not re-run; its hypervisor, `wdt-b` cell and probes are the same.

In the earlier runs, case 2 loaded `wdt-probe-4.bin`, the build with UART output, into `wdt-b`. The probe's first UART access was a stage-2 fault (`Unhandled data read at 0x11001214(4)`), so `wdt-b` failed with CPU 5 under "Failed CPUs" before its watchdog could expire. This affects the runs of 2026-10-22 (board logs 157 and 166) and the v26.0 dry run (board log 152c), where case 2 did not test the watchdog. Case 1 was not affected.

Sources. `rity-w1.sh`, stage TWO, and `m3-two.sh`; board log 188; board log 106; board log 157, board log 166.

3.6 Stage D: the restart daemon

The daemon `jailhouse-cell-restartd` waits for cells to fail, through `poll()` on their state in sysfs and a 5 s fallback. It then destroys, creates, loads and starts the failed cell again, as its configuration says. For a cell with a remote processor, it first detaches the remoteproc.

- W1-D0 uses the image's unit as it is installed.
- W1-D1 to W1-D6 run the image's daemon binary as a transient systemd unit, with a configuration in `/root`.

W1-D0 The shipped unit restarts every cell type

Purpose. The daemon and unit that the image installs must restart every kind of cell that the image ships. This includes the rpmsg cell, whose remoteproc Linux must detach first.

Procedure.

1. Write a configuration with all seven Zephyr cell types: `restart_limit = 3` in `restart_window = 60`, restart image probe 6, and `remoteproc = 0001:00:00.0` for `-zephyr-rpmsg`.
2. Install it as `/etc/jailhouse/cell-restart.conf` and restart the image's unit `jailhouse-cell-restartd`.
3. For each cell type: start the cell with probe 2 (software reset after 1 s). After 5 s, list the cell and read UART1. Destroy the cell.
4. Print the daemon's journal. Then stop the unit and remove the configuration.

Checks (W1 milestone notes).

- All seven types fail through the software reset, and the daemon restarts each with probe 6, which reads `s40000000`.
- Each restart takes 0.15-0.34 s.
- For the rpmsg cell, the daemon detaches the remoteproc before it restarts the cell: `remoteproc 0001:00:00.0: detached` comes between "restarting" and "restarted".

Expected output. For each cell type, for example `-zephyr`:

```
--- 11:50:16 zephyr
start rc=0
1       zephyr                  running           3
UART1: 2s00000000wR6s40000000k00000000i00000000t40000000
```

The journal (board clock):

```
Jun 26 09:01:28.056795 [...]: restarting zephyr, zephyr-a78, zephyr-afe, zephyr-afe-a78, zephyr-
afe-smp, zephyr-rpmsg, zephyr-smp when they fail
Jun 26 09:01:33.063202 [...]: zephyr: restarting
Jun 26 09:01:33.216511 [...]: zephyr: restarted
[...]
Jun 26 09:02:14.624083 [...]: zephyr-rpmsg: restarting
Jun 26 09:02:14.625924 [...]: remoteproc 0001:00:00.0: detached
Jun 26 09:02:14.743385 [...]: zephyr-rpmsg: restarted
```

Last run. As the criteria on both boards: all seven types restarted (board logs 157 and 166).

Sources. `rity-w1.sh`, stage D; board log 157.

W1-D1 Back-off and restart limit

Purpose. A cell that fails again and again must not be restarted in a tight loop. The daemon backs off, and it gives up after a limit, so that a broken image does not keep the system busy.

Procedure. Configuration: probe 4 (fails 3 s after each start), limit 5 in 300 s. Start the cell, wait 60 s, and print the journal.

Checks (W1 milestone notes).

- The restarts come 1, 3, 7 and 15 s after the second to fifth failures.
- The sixth failure gives up at once: "failed 5 times within 300 s, not restarted until it runs again".

Expected output.

```
Jun 26 09:02:24.392275 [...]: zephyr: restarting
Jun 26 09:02:24.498660 [...]: zephyr: restarted
Jun 26 09:02:28.529882 [...]: zephyr: restarting
[...]
Jun 26 09:02:34.730887 [...]: zephyr: restarting
[...]
Jun 26 09:02:44.931472 [...]: zephyr: restarting
[...]
Jun 26 09:03:03.135428 [...]: zephyr: restarting
Jun 26 09:03:03.250518 [...]: zephyr: restarted
Jun 26 09:03:06.327939 [...]: zephyr: failed 5 times within 300 s, not restarted until it runs
again
```

The first restart can come up to 5 s after the first failure: a cell created while the daemon waits is watched only from the daemon's next wake-up. The W1 milestone notes record this as a known limit.

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage D; board log 157.

W1-D2 Ten expiry and restart cycles

Purpose. Repeated failures and restarts must not leak hypervisor memory or CPUs.

Procedure. Configuration: probe 4, limit 20 in 4 s. Read the hypervisor's pool counters and the online CPUs. Start the cell, wait for ten "restarted" lines in the journal, stop the daemon, destroy the cell, and read the counters again.

Checks (W1 milestone notes). Ten restarts, one every 4.0-4.3 s. `mem_pool_used` and `remap_pool_used` are the same before and after, and all CPUs are online.

Expected output.

```
before: mem_pool_used 96 remap_pool_used 532 online 0-7
[...]
10 restarts
[...]
destroy rc=0
after: mem_pool_used 96 remap_pool_used 532 online 0-7
```

Last run. As the criteria: 78 and 532 pages on the 510, 96 and 532 on the 700, before and after (board logs 157 and 166).

Sources. `rity-w1.sh`, stage D; board log 157.

W1-D3 Restart after a stage-2 fault

Purpose. The daemon must also restart a cell that a fault has stopped. The cause in `STA` must stay clear, because a fault is not a watchdog cause.

Procedure. Configuration: restart image probe 6. Start the cell with probe 9.

Checks (W1 milestone notes). The daemon restarts the cell with probe 6, which reads `s00000000`.

Expected output.

```
UART1: 9s00000000X6s00000000k00000000i00000000t00000000
Jun 26 09:04:09.975181 [...]: zephyr: restarting
Jun 26 09:04:10.099550 [...]: zephyr: restarted
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage D; board log 157.

W1-D4 The rpmsg cell: detach, restart and attach again

Purpose. When an rpmsg cell fails, Linux's side of the channel must be detached before the restart, and must attach again to the new instance. Otherwise Linux's rpmsg devices would hang on a dead peer.

Procedure.

1. Configuration: `-zephyr-rpmsg` with the rpmsg sample image and `remoteproc = 0001:00:00.0`.
2. Start the cell. Run `rpmsg-test` for 8 s; it is stopped by `SIGINT`, so its return code is 124. Print the remoteproc state before and after.
3. Load probe 2 into the attached cell and start it in place. After 1 s, the probe resets the cell.
4. Print the journal, and run `rpmsg-test` again.

Checks (W1 milestone notes).

- `rpmsg-test` attaches and receives "Hello from Zephyr! 2, 3".
- After the reset, the daemon detaches the remoteproc, then destroys, creates, loads and starts the cell.
- `rpmsg-test` then reports the state "detached" before and "attached" after, and receives the messages again.

Expected output.

```
state before: detached
rpmsg-test rc=124
[INFO]  send_message  Size: 21
[INFO]  RPMSG RX  len: 22  msg: 'Hello from Zephyr!  2'
[INFO]  send_message  Size: 21
[INFO]  RPMSG RX  len: 22  msg: 'Hello from Zephyr!  3'
state after: attached
--- 11:53:12 in place: probe 2 (SWRST after 1 s) into the attached rpmsg cell
[...]
Jun 26 09:04:25.926143 [...]: zephyr-rpmsg: restarting
Jun 26 09:04:25.927786 [...]: remoteproc 0001:00:00.0: detached
Jun 26 09:04:26.123531 [...]: zephyr-rpmsg: restarted
state before: detached
rpmsg-test rc=124
[...]
[INFO]  RPMSG RX  len: 22  msg: 'Hello from Zephyr!  3'
state after: attached
```

Last run. As the criteria on both boards (board logs 157 and 166).

Sources. `rity-w1.sh`, stage D; board log 157.

W1-D5, W1-D6 Zephyr's warm and cold reboot

Purpose. A Zephyr application reboots with `sys_reboot()`. A warm reboot (D5) uses PSCI `SYSTEM_RESET2`, a cold one (D6) `SYSTEM_RESET`. In a cell with a watchdog, both must fail the cell, and the daemon must restart it. Both calls must be reported as supported.

Procedure. For each kind:

1. Configuration: restart image `hwinfo_api`.
2. Start `-zephyr` with the reboot test application (`reboot-warm` or `reboot-cold`). The application prints its reset cause, the supported causes and `PSCI_FEATURES` of both calls, then reboots.
3. After 10 s, list the cells and print the journal.

Checks (the RITY move notes).

- The application's first line is `CAUSE 0x8 (RESET_POR)`: the cause of its first start, not of the reboot.
- `PSCI_FEATURES` reports both calls as supported (0).
- The cell fails with "system reset requested", and the daemon restarts it.
- `hwinfo_api` passes.

Expected output. D5:

```
*** Booting Zephyr OS build 2e9e70f0a16c ***
CAUSE 0x8 (0): RESET_POR
SUPPORTED 0x1a (0)
PSCI_FEATURES SYSTEM_RESET 0x84000009 = 0
PSCI_FEATURES SYSTEM_RESET2 0xc4000012 = 0
WARM REBOOT
*** Booting Zephyr OS build 2e9e70f0a16c ***
Running TESTSUITE hwinfo_device_id_api
[...]
PROJECT EXECUTION SUCCESSFUL
```

```
Cell "zephyr" failed: system reset requested
[...]
Jun 26 09:04:58.714185 [...]: zephyr: restarting
Jun 26 09:04:58.883801 [...]: zephyr: restarted
```

D6 prints `COLD REBOOT` instead of `WARM REBOOT`, and is otherwise the same.

The restarted image is `hwinfo_api`, which does not print the cause. So these runs do not show the cause after the reboot. The cause after a warm reboot, `RESET_SOFTWARE`, is shown only in board log 136 (REL-10).

Last run. As the criteria on both boards (board logs 157 and 166). The script ends with the kernel's warning count, 0, and with only the root cell running.

Sources. `rity-w1.sh`, stage D; board log 157.

04
Release-level checks

4.1 Overview
4.2 Samples and tools
4.3 Boot without the overlay
4.4 Real resets of the SoC
4.5 Checks of specific changes

4.1 Overview

These scripts cover what the regression does not: the samples and tools that customers use, the boot without the overlay, the SoC's real resets, and fixes with a test of their own. Each runs from the host over adb. One of them prints a verdict: `fixes2-test.sh` prints `good` or `BAD` per reload. The others print observations, judged against the criteria given in the Checks field, which come from the scripts' headers and the RITY move notes.

Some scripts deploy a hypervisor build of their own before the test: `pcie-test.sh`, `sr2-test.sh` and `fixes2-test.sh` copy a build's `jailhouse.bin`, module and cells to the board. They ran on IoT Yocto v26.0 with the build named in each entry, not with an image that ships Jailhouse. When both boards are on adb, these three scripts take the adb transport from the variable `ADB_T`.

4.2 Samples and tools

REL-01 The rpmsg sample

Purpose. The rpmsg sample of the Genio Zephyr samples (`samples/rpmsg`) is the documented way for Linux and a Zephyr cell to exchange messages:

- Zephyr runs in `-zephyr-rpmsg`;
- Linux attaches to the cell as a remote processor over the shared-memory (ivshmem) device of the virtual PCI host;
- `rpmsg-test` sends messages both ways.

The test shows that the whole path works on the image: the driver `mtk_jh_rproc`, virtio, the rpmsg character devices and the cell.

Procedure. `rpmsg-sample.sh <board> <zephyr.bin> <rpmsg-test> <prefix> [secs]`:

1. Push the Zephyr image and `rpmsg-test`, and print their md5.
2. Print the kernel options, the remoteprocs and the `/dev/rpmsg*` nodes. Find the ivshmem remoteproc by its name, `0001:00:00.0`.
3. Create and start `-zephyr-rpmsg`. After 3 s, run `rpmsg-test -p <n>` for 20 s. Print the remoteproc state before, during and after, the device nodes and the modules.
4. Detach the remoteproc, destroy the cell, and print the end of the hypervisor console. Count the messages from Linux on UART1.

Checks. No coded verdict. The criteria (RITY move notes):

- the remoteproc attaches;

- 9 messages from Linux arrive;
- `rpmsg-test` returns 0;
- the remoteproc detaches, the cell is destroyed, and all CPUs are back.

Expected output. The 700 on 2026-10-26, abridged:

```
ivshmem remoteproc id: 1
=== 11:20:07 Zephyr cell
rc=0
=== 11:20:10 rpmsg-test -p 1, 20s
state before: detached
state while running: attached
crw------- 1 root root 507, 0 Jun 26 08:46 /dev/rpmsg0
crw------- 1 root root 506, 0 Jun 26 08:46 /dev/rpmsg_ctrl0
[...]
rpmsg-test rc=0
[...]
=== 11:20:37 detach and destroy zephyr-rpmsg
state detached
destroy rc=0
0-7
=== hypervisor console (tail)
Shared memory connection established, peer cells:
 "genio-700-evk"
Created cell "zephyr-rpmsg"
[...]
=== UART1: 9 messages from Linux
[...]
[INFO]  Created endpoint for service 'rpmsg-raw'
[INFO]  Endpoint for service 'rpmsg-raw' is connected
```

Last run. As the criteria on both boards, 2026-10-26, refinement-1 image, as step 2 of IMG-03 (board logs 178 and 186). Also on 2026-10-22 on the heads image (board logs 158 and 167).

Sources. `rpmsg-sample.sh`; board log 186, board log 158.

REL-02 The samples' cell launcher `genio-inmate.sh`

Purpose. The samples provide `tools/genio-inmate.sh`, which customers use to run a Zephyr image in a cell with one command. It must work from cold (enabling Jailhouse itself), with the default and with a named cell, and when it runs over a cell that is already running.

Procedure. `genio-inmate-test.sh <board> <prefix>`, with the launcher of samples `b962ff3` and `hello_world`:

1. Destroy any cell, disable Jailhouse, and unload the module.
2. Run the launcher three times, capturing UART1 for 5 s each: with the default cell, then with `genio-<board>-evk-zephyr-a78.cell` twice. The second A78 run replaces a running cell.
3. Destroy every cell, disable Jailhouse, and unload the module.

Checks. No coded verdict. The criteria (RITY move notes): from cold, the launcher enables the root cell itself; each run returns 0, the cell is listed running, and UART1 shows "Hello World".

Expected output. The 700, abridged:

```
=== 11:55:22 from cold
0       genio-700-evk           running           0-7
module loaded: 0
=== 11:55:22 run 1: default cell
[...]
1       zephyr                  running           3
rc=0
UART1: Hello World! mt8390_genio_700_evk/mt8188/a55
=== 11:55:27 run 2: genio-700-evk-zephyr-a78.cell
[...]
1       zephyr-a78              running           7
rc=0
UART1: Hello World! mt8390_genio_700_evk/mt8188/a55
=== 11:55:32 run 3: genio-700-evk-zephyr-a78.cell
[...]
rc=0
UART1: Hello World! mt8390_genio_700_evk/mt8188/a55
[...]
disable rc=0
online 0-7
```

Last run. As the criteria on both boards, 2026-10-22, heads image (board logs 159 and 168).

Sources. `genio-inmate-test.sh`; board log 159.

REL-03 Refusal of the root cell by a hypervisor without the MediaTek units

Purpose. The Genio root cells carry vendor resources (the MediaTek entries) that only the MediaTek units handle. A hypervisor built without `CONFIG_SOC=mt8188` must refuse such a cell, rather than run it without mediation. Linux must survive the refusal.

Procedure. `extras.sh <board> <log prefix> [no-SoC jailhouse.bin]`, part 1. The script uses the image's own Jailhouse files, and leaves Jailhouse as it found it (enabled or not, module loaded or not). It stops if it does not see exactly one adb device, or if the board is not the one named.

1. Push a hypervisor built at the image's commit without `CONFIG_SOC` (a build of the image's commit without `CONFIG_SOC`). Without this argument, part 1 is skipped.
2. Disable Jailhouse and unload the module. Bind-mount the build over `/usr/lib/firmware/jailhouse.bin`, so that the image's file is never written. Then `modprobe jailhouse` and `jailhouse enable`.
3. Print the result, the console's start-up and error lines, and Linux's uptime.
4. Unload the module, unmount the build, check the image's file, and enable again.

Checks. No coded verdict. The enable fails, the console names the reason, the hypervisor is not enabled, and Linux stays up. After the restore, the image's hypervisor enables with the MediaTek units.

Expected output. Board log 190, the 700:

```
=== 13:17:18 (1) refusal: decfbb49, built without CONFIG_SOC
disable rc=0
decfbb49 /usr/lib/firmware/jailhouse.bin
JAILHOUSE_ENABLE: Invalid argument
enable rc=1
enabled 0
Initializing Jailhouse hypervisor mtk-v1.0.0 (28-ge7fd8796) on CPU 6
Initializing processors:
Initializing unit: irqchip
Initializing unit: ARM SMMU v3
Initializing unit: ARM SMMU
Initializing unit: PVU IOMMU
Initializing unit: PCI
ERROR: Cell "genio-700-evk" has vendor resources, but no unit handles them
Linux alive: 10:43:29 up 1:59, 0 user, load average: 0.44, 0.17, 0.06
--- restore
688a5889 /usr/lib/firmware/jailhouse.bin
0
enable rc=0
Initializing Jailhouse hypervisor mtk-v1.0.0 (28-ge7fd8796) on CPU 7
Initializing unit: mt8188_clk
Initializing unit: mt8188_eint
Initializing unit: mt8188_gpio
Initializing unit: mt8188_wdt
```

The `0` after the restore is the count of remaining `jailhouse.bin` mounts.

Last run. As described, on the 700, 2026-10-26, refinement-1 image (board log 190). Before, the script's earlier form ran on 2026-10-08 with the release 1.0 files (board log 78); it replaced the firmware file instead of mounting over it.

Sources. `extras.sh`; board log 190, board log 78.

REL-04 Disable and enable cycles with the ivshmem device

Purpose. With the shared-memory device on the virtual PCI host, a disable must remove it cleanly, and an enable must bring it back with its remoteproc. Leaks here would show as kernel warnings, or as remoteprocs that are never freed.

Procedure. `extras.sh`, part 2. Print the ivshmem device's IDs, its remoteproc and the kernel's warning count. Then three times: `jailhouse disable`, check that the device `0001:00:00.0` is gone, `jailhouse enable`, and print the device's IDs, the remoteproc, its state and the warning count.

Checks. No coded verdict. In each cycle the disable and the enable return 0, the device is gone after the disable and back after the enable (`0x110a 0x4106`), its remoteproc is bound again, and the kernel's warning count stays 0.

Expected output. Board log 190, the 700:

```
=== 13:17:18 (2) three disable/enable cycles with the ivshmem device
before: ivshmem 0x110a 0x4106 , remoteproc remoteproc1 detached, kernel warnings 0
cycle 1: disable=0 ivshmem gone after disable=yes enable=0 ivshmem 0x110a 0x4106  remoteproc
remoteproc1 detached kernel warnings 0
cycle 2: disable=0 ivshmem gone after disable=yes enable=0 ivshmem 0x110a 0x4106  remoteproc
remoteproc1 detached kernel warnings 0
cycle 3: disable=0 ivshmem gone after disable=yes enable=0 ivshmem 0x110a 0x4106  remoteproc
remoteproc1 detached kernel warnings 0
```

Last run. As described, on the 700, 2026-10-26, refinement-1 image (board log 190). Before: 2026-10-08, release 1.0 files (board log 78).

Sources. `extras.sh`; board log 190.

REL-05 `hello_world` in the rpmsg cell

Purpose. The rpmsg cell must also run an ordinary Zephyr image, and the hypervisor must set up its shared-memory connection to the root cell.

Procedure. `extras.sh`, part 3. Run `hello_world` in `-zephyr-rpmsg` for 6 s, with UART1 captured. Print the cell list, the end of the hypervisor console, the remoteproc's state and the UART1 output, then destroy the cell. At the end, print the cell list, the ivshmem device and the kernel's warning count, and restore Jailhouse's state as found.

Checks. No coded verdict. The cell runs, the console reports "Shared memory connection established" with the root cell as peer, UART1 shows "Hello World", the destroy returns 0, and there are no kernel warnings.

Expected output. Board log 190, the 700:

```
=== 13:17:25 (3) hello_world in genio-700-evk-zephyr-rpmsg.cell
start rc=0
0       genio-700-evk           running           0-2,4-7
1       zephyr-rpmsg            running           3
--- hypervisor console (tail)
[...]
Adding virtual PCI device 00:00.0 to cell "zephyr-rpmsg"
Shared memory connection established, peer cells:
 "genio-700-evk"
Created cell "zephyr-rpmsg"
[...]
--- UART1
*** Booting Zephyr OS build 2e9e70f0a16c ***
Hello World! mt8390_genio_700_evk/mt8188/a55
destroy rc=0
=== 13:17:33 after
0       genio-700-evk           running           0-7
ivshmem 0x110a 0x4106 remoteproc remoteproc1 detached
kernel warnings this boot: 0
```

Last run. As described, on the 700, 2026-10-26, refinement-1 image (board log 190). Before: 2026-10-08, release 1.0 files (board log 78).

Sources. `extras.sh`; board log 190.

4.3 Boot without the overlay

REL-06 Booting without `jailhouse.dtbo`, and restoring it

Purpose. The Jailhouse overlay reserves the hypervisor's memory and the ivshmem window. An image that ships Jailhouse must still boot as a stock board without the overlay: all memory goes to Linux, and Jailhouse cannot be enabled, with the documented message. Putting the overlay back must restore everything. This is the switch that customers use.

Procedure. `no-overlay.sh <prefix> off`, then `no-overlay.sh <prefix> restore`.

- **off:**
  1. Save `boot_conf` and `list_dtbo` from the U-Boot environment to the host and the board.

2. Remove `jailhouse.dtbo` from both, after checking that it occurs exactly once, and read them back.
3. Reboot, waiting for a new boot ID.
4. Print the memory map and the reserved-memory nodes, and try `jailhouse enable`.

- **restore:** set both variables back from the saved copy, reboot, print the same state, enable, then disable.

Checks (the script's header, no coded verdict).

- **Off:** System RAM spans `0x6ac00000-0x6bb00000` without a gap; there is no `memory@6ba00000` node; the enable fails with "request_mem_region failed for hypervisor memory".
- **Restore:** the gap and the reservation are back, and the enable works.

Expected output. The 700, abridged. Without the overlay:

```
--- without the overlay
boot_conf=#conf-mediatek_mt8390-genio-700-evk.dtb#conf-video.dtbo#conf-apusys.dtbo#conf-display-
dsi.dtbo#conf-display-hdmi.dtbo
[...]
60000000-617fffff : reserved
61800000-23ffffff : System RAM
  66fc0000-67d6ffff : reserved
memory@6ba00000 nodes: 0
[...]
JAILHOUSE_ENABLE: Invalid argument
enable rc=1
[   21.417969] jailhouse: request_mem_region failed for hypervisor memory.
[   21.417976] jailhouse: Did you reserve the memory with "memmap=" or "mem="?
```

Restored:

```
--- restored
[...]
61800000-6abfffff : System RAM
  66fc0000-67d6ffff : reserved
6ba00000-6baffffff : reserved
6bb00000-23ffffff : System RAM
memory@6ba00000 nodes: 1
[...]
enable rc=0
[   21.089549] The Jailhouse is opening.
0       genio-700-evk           running           0-7
disable rc=0
```

Last run. As the criteria on both boards, 2026-10-22, heads image (board logs 160 and 169). Without the overlay, System RAM spans `61800000-13ffffff` on the 510 and `61800000-23ffffff` on the 700.

Sources. `no-overlay.sh`; board log 160.

4.4 Real resets of the SoC

Both tests use `reset-genio.sh <a|b> <console tty> <prefix>`, with UART0 on the host at 921600 baud. The script:

1. Prints the boot ID, the module state and the watchdog's `MODE`, `LENGTH` and `STA` before and after the reset.
2. Waits up to 240 s for adb to come back with a new boot ID.
3. Has the console script `console.py` stop U-Boot, run `md.l 0x10007000 4` (`MODE`, `LENGTH`, `RESTART`, `STA`), and continue with `run bootcmd`.

4. Prints the kernel's watchdog lines and its warning count, and the console lines around the reset.

A reset-cause check needs a power-cycled start. On the 510, `STA` does not accumulate a watchdog reset on top of an earlier software reboot (board logs 171 and 173).

REL-07 Hardware watchdog reset after a Linux crash

Purpose. If Linux crashes while cells run, nothing kicks Linux's watchdog. The SoC's hardware watchdog must then reset the whole board within its 31 s timeout, and the firmware must report the hardware watchdog as the cause. The cells' kicking must not keep the SoC alive: their watchdogs are virtual.

Procedure. Test `a`:

1. Push `wdt-probe-10-nouart.bin` and the `wdt-b` cell, and `sync`. A crash comes before writeback, so unsynced files come back corrupt.
2. Enable Jailhouse. Start `-zephyr` and `wdt-b`, each with probe 10 (kicking every second).
3. Set the console log level to 8, then crash Linux with `echo c > /proc/sysrq-trigger`. The kernel does not reboot on panic.

Checks (the script's header and the RITY move notes, no coded verdict).

- The SoC resets within about 31 s of the crash.
- BL2 prints "WDT: Status = 0x80000000" and "Last reset was hardware watchdog".
- U-Boot reads `STA 0x80000000`, and so does Linux after the boot.
- The new boot has 0 kernel warnings.

Expected output. The 700 on 2026-10-26, abridged:

```
=== 11:07:18.769 trigger: echo c > /proc/sysrq-trigger
=== 11:07:58 adb back with a new boot (f83cf4d8-ddb8-44ea-baad-3a1cbbc1d4b2), 18 s of waiting
[...]: first line of the new boot at 29.798 s
[...]
0x1000700c: 0x80000000
[...]
kernel warnings this boot: 0
=== console, around the reset
2:   1.002 [   54.014152] sysrq: Trigger a crash
3:   1.002 [   54.014613] Kernel panic - not syncing: sysrq triggered crash
[...]
31:  29.798 NOTICE:  BL2: Built : 03:14:36, Oct 14 2026
32:  29.798 NOTICE:  WDT: Status = 0x80000000
33:  29.798 NOTICE:  WDT: Last reset was hardware watchdog
[...]
226:  32.353 => md.l 0x10007000 4
227:  32.353 10007000: 00000004 0000ffe0 00000000 80000000  ................
```

Last run. As the criteria:

- the 700 on 2026-10-26, heads image: reset about 29 s after the crash (board log 181);
- the 510 on 2026-10-22, heads image, from a power-cycled start: reset about 31 s after the crash (board log 173).

On the 510 the first attempt, board log 171, started with bit 30 already latched, and BL2 reported a software reboot.

Sources. `reset-genio.sh`, `console.py`; board log 181, board log 173.

REL-08 Plain reboot and the reset status

Purpose. A normal `reboot` from Linux must work on the image, and it records what the firmware reports for a software reboot. A reset-cause check in a product depends on this.

Procedure. Test `b`:

1. Unload Jailhouse, if it is loaded.
2. Add kprobes on `ksys_sync`, `psci_sys_reset` and `mtk_wdt_restart`, printed to the console.
3. `reboot`.

Checks (the script's header and the RITY move notes, no coded verdict). The board comes back, and BL2, U-Boot and Linux report `STA`. The kprobe lines need the boot argument `tp_printk`; without it they do not appear (`no jh_sync line in dmesg`).

Expected output. The 700 on 2026-10-26, abridged. The 700 adds bit 30 to bit 31, latched by REL-07, which ran just before:

```
=== 11:08:11.202 trigger: reboot
=== 11:08:27 adb back with a new boot (612de2f3-30bb-4fd5-8f36-1cf7e7ec79dd), 16 s of waiting
[...]
0x1000700c: 0xc0000000
[...]
kernel warnings this boot: 0
=== console, around the reset
128:   3.507 [   22.173069] reboot: Restarting system
[...]
131:   4.509 NOTICE:  WDT: Status = 0xc0000000
132:   4.509 NOTICE:  WDT: Last reset was hardware watchdog
[...]
326:   7.064 10007000: 00000004 0000ffe0 00000000 c0000000  ................
```

Last run.

- The 700 on 2026-10-26: back after 16 s, `STA 0xc0000000` (board log 182).
- The 510 on 2026-10-22: back after 14 s; BL2 and U-Boot read `STA 0x40000000`, "Last reset was normal software reboot" (board log 172).

Sources. `reset-genio.sh`; board log 182, board log 172.

4.5 Checks of specific changes

REL-09 PCIe controller unbind and rebind with cells running (700)

Purpose. The watchdog unit passes Linux's subsystem resets through `SWSYSRST` and keeps the bits that a cell owns (W1-M2-T10). The test was written to drive that root path from a real Linux driver: the PCIe controller, whose power-down and power-up were expected to toggle its reset bit 19. On this BSP it does not reach `SWSYSRST`, as the script's header says since 2026-10-26 (see Last run). What it shows is that a PCIe rebind while cells run works and produces no hypervisor fault.

Procedure. `pcie-test.sh <build dir> <prefix>`, on the 700:

1. Deploy the build's hypervisor (`a0b442e8`), module and cells, and enable.
2. Find the PCIe controller behind the mt7921 Wi-Fi (`112f0000.pcie`, driver `mtk-pcie-gen3`).
3. With `-zephyr`, then with `-zephyr-afe` running: unbind and rebind the controller. Print `SWSYSRST`, the PCI devices and the network link before, in between and after.

4. Disable and enable. Print the kernel log since the start and the hypervisor's `WDT`, `Unhandled` and `FATAL` lines.

Checks. No coded verdict. The devices re-enumerate, the cells keep running, and the hypervisor console has no `WDT`, `Unhandled` or `FATAL` line.

Expected output. Abridged:

```
=== 18:14:03 with zephyr running
--- before
SWSYSRST 0x10007018: 0
0000:00:00.0 PCI bridge [0604]: MEDIATEK Corp. Device [14c3:8188] (rev 01)
0000:01:00.0 Network controller [0280]: MEDIATEK Corp. MT7921 802.11ax PCI Express Wireless
Network Adapter [14c3:7961]
[...]
unbind rc=0
SWSYSRST unbound 0x10007018: 0
PCI devices left: 0
bind rc=0
--- after rebind
SWSYSRST 0x10007018: 0
0000:00:00.0 PCI bridge [0604]: MEDIATEK Corp. Device [14c3:8188] (rev 01)
0000:01:00.0 Network controller [0280]: MEDIATEK Corp. MT7921 802.11ax PCI Express Wireless
Network Adapter [14c3:7961]
[...]
kernel warnings: 0 before, 7 now
hypervisor WDT|Unhandled|FATAL lines: 0
```

Last run. 2026-10-12, on the 700 (board logs 140 and 141): pass, with two findings.

- `SWSYSRST` reads 0 throughout. On this BSP no device-tree node consumes the TOPRGU's resets, and `mtk-pcie-gen3` resets its endpoint with the controller's own PERST#. **The test does not exercise the root path of `SWSYSRST`.** W1-M2-T10 covers that path with direct writes.
- The kernel warnings in `mtk_pcie_probe`, the IOMMU group failures and the Wi-Fi firmware time-outs occur in the same way with Jailhouse disabled (log 141). They come from the BSP's driver.

Sources. `pcie-test.sh`; board log 140, board log 141.

REL-10 PSCI `SYSTEM_RESET2` (510)

Purpose. Zephyr's warm reboot uses PSCI `SYSTEM_RESET2`. Before this change, `PSCI_FEATURES` returned −1 (`NOT_SUPPORTED`) for both reset calls, Zephyr's warm reboot ended in "Failed to reboot: spinning endlessly", and the cell kept running. With the change:

- for a cell with a watchdog, `PSCI_FEATURES` reports both calls, and the warm reset fails and restarts only the cell, like `SYSTEM_RESET`;
- for a cell without one, both calls answer `NOT_SUPPORTED`;
- an invalid reset type is refused.

Procedure. `sr2-test.sh <build dir> <prefix> [warm app] [hwinfo app]`, on the 510, with the build `a0b442e8`. Each case is one cell run of 1-2 s, with UART1 captured. `hello_world` runs first, to set up UART1.

Case	Cell and image	What it shows
T1	`-zephyr`, probe 11	`PSCI_FEATURES` of both calls, `SYSTEM_RESET2` of an invalid type (1), then the warm reset
T2	`-zephyr`, probe 6	`STA` of the next cell
T3	`-uart-demo`, probe 11 without the `STA` read, at address 0	The same calls in a cell without a watchdog
T4	`-zephyr`, probe 5, then probe 6	`SYSTEM_RESET`, then `STA`, as before the change
T5	`-zephyr`, the warm-reboot application, with the daemon	Zephyr's `sys_reboot(SYS_REBOOT_WARM)` and the restart

Checks (the script's header, no coded verdict).

- **T1:** `PSCI_FEATURES` returns 0 for both calls; type 1 returns −2 (`INVALID_PARAMETERS`); the warm reset does not return and fails the cell with "system reset requested".
- **T2:** the next cell reads `STA 0x40000000`.
- **T3:** every call returns −1 (`NOT_SUPPORTED`), and the cell keeps running.
- **T4:** as W1-M3-T13.
- **T5:** the warm reboot fails the cell with "system reset requested", the daemon (restart limit 2) restarts it with the restart image, and `hwinfo_api` then reads the cause `RESET_SOFTWARE` and passes. T5 needs both applications, and since 2026-10-26 it is not run if the restart image is the warm-reboot application itself. Its heading names the restart image, and the daemon's configuration is kept beside the log (`<prefix>-t5.conf`).

Expected output. With the change (board log 136):

```
=== 18:03:32 T1-system-reset2
1       zephyr                  failed          3
Cell "zephyr" failed: system reset requested
UART1: Rs00000000f00000000g00000000effffffe
=== 18:03:36 T2-sta
1       zephyr                  running         3
[...]
UART1: 6s40000000k00000000i00000000t40000000
=== 18:03:39 T3-no-watchdog
1       uart-demo               running         3
[...]
UART1: RfffffffffgfffffffeffffffffFffffffff
=== 18:03:43 T4-system-reset
1       zephyr                  failed          3
[...]
UART1: 5s00000000
```

T5:

```
21:36:50.671374 zephyr: restarting
21:36:50.785235 zephyr: restarted
[...]
UART1:
*** Booting Zephyr OS build 278f7e212e62 ***
CAUSE 0x8 (0): RESET_POR
PSCI_FEATURES SYSTEM_RESET 0x84000009 = 0
PSCI_FEATURES SYSTEM_RESET2 0xc4000012 = 0
WARM REBOOT
*** Booting Zephyr OS build 278f7e212e62 ***
CAUSE 0x2 (0): RESET_SOFTWARE
[...]
kernel warnings this boot: 0
```

Last run. 2026-10-12, on the 510.

- Board log 135: T1 to T4.
- Board log 136: T1 to T5, as the criteria. The restarted cell reports `CAUSE 0x2 (0): RESET_SOFTWARE`.
- Board log 137: the reference before the change (`82476e42`).

Log 136 was run with the warm-reboot application as both arguments, as the script's header now records. So the restarted image was the same application: it printed its own lines (`CAUSE`, `PSCI_FEATURES`, `WARM REBOOT`) and rebooted again, until the daemon's limit of 2 stopped it. That run showed the restart limit, and the cause after a warm reboot, `RESET_SOFTWARE`, as the application's own `hwinfo` call reads it. It is the only log that shows the cause after a warm reboot (W1-D5 shows the cause of the first start). T5 has not run with `hwinfo_api` as the restart image.

Sources. `sr2-test.sh`; board log 135, board log 136, board log 137.

REL-11 In-place reload runs the new image (700)

Purpose. A cell can be loaded again and started without a destroy, for example by the restart daemon after a failure. The CPU must then run the new image. On the Genio 700, before fix 2 of release 1.1, the reloaded cell did not run the new image, because of stale contents of the instruction cache.

Procedure. `fixes2-test.sh <build dir> <prefix> [iterations]`, part IC, on the 700, with the build `dd9cec90`. `ttyS1` is kept active, so that UART1 stays clocked. Ten times:

1. Probe 4 times out in `-zephyr`.
2. Load probe 6 into the same failed cell and start it, without a destroy.
3. Check UART1 and the console's `FATAL` count.

Checks. Coded: a reload is `good` if UART1 shows probe 6's `6s80000000k00000000` and the console has no new `FATAL` line. The script prints the count of good reloads.

Expected output. Board log 126:

```
=== IC: 10 in-place reloads, probe 4 (times out) then probe 6 in the same cell
1: good (UART1 "6s80000000k00000000i00000000t80000000", FATAL lines 0 -> 0)
2: good (UART1 "6s80000000k00000000i00000000t80000000", FATAL lines 0 -> 0)
[...]
IC: 10 of 10 in-place reloads ran the new image
```

Before the fix (board log 125):

```
1: BAD (UART1 "", FATAL lines 0 -> 2)
```

Last run. 10 of 10 good on the 700, 2026-10-12 (board log 126), against 0 of 10 before the fix (board log 125). The W1 cases W1-M3-T11 (in place) and W1-D0 repeat the in-place reload on every W1 run.

Sources. `fixes2-test.sh`; board log 125, board log 126.

REL-12 The base of a vendor entry (700)

Purpose. The GPIO and EINT entries of a cell configuration cover a range of pins from a base. The hypervisor accepts only a base that is a multiple of 32, up to 128. It must refuse any other base when the cell is created, with a message that names the cell and the rule.

Procedure. `fixes2-test.sh`, part BASE: create the test cells `test-base36` (GPIO entry base 36) and `test-base32` (base 32), print the console and the cell list, and destroy what was created. The test cells are built from sources of their own, whose `build.sh` rebuilds them, and the regression's test cells, byte for byte.

Checks. No coded verdict. `test-base36` is refused with the console's reason, and `test-base32` is created.

Expected output.

```
=== BASE: test-base36 (GPIO base 36) and test-base32 (base 32)
JAILHOUSE_CELL_CREATE: Invalid argument
test-base36 create rc=1
[...]
ERROR: Cell "test-base36": vendor entry of type 2 starts at 36, which is not a multiple of 32 up
to 128
test-base32 create rc=0
Created cell "test-base32"
[...]
kernel warnings this boot: 0
```

Last run. As the criteria on the 700, 2026-10-12 (board log 126).

Sources. `fixes2-test.sh`, the test cells' `build.sh`; board log 126; the script-fix record (Section 1.5).

05
Image and flash checks

These checks go with every flash of an image that ships Jailhouse: a gate before the flash, a first-boot check after it, and an acceptance test for an image that changes the kernel configuration. They ran for the heads image on 2026-10-22 and for the refinement-1 image on 2026-10-26. The expected output is from the 700's refinement-1 flash.

IMG-01 Pre-flight gate

Purpose. `genio-flash` takes the image named in the deploy directory's `rity.json`. Another image built in the same directory rewrites that file: on 2026-10-22 a `rity-demo-image` build did so. Flashing the wrong image, or an image without the overlay, wastes a board session and can lose the board's state. The gate stops before that happens.

Procedure. `rity-precheck.sh <board> <prefix>`, on the host:

1. Read `mmc0` in `rity.json`.
2. Run `genio-flash -i rity-bringup-image --load-dtbo jailhouse.dtbo --dry-run`.
3. Compare the md5 of the wic with the build record (or `WIC_EXPECT`), and the FIT's with `FIT_EXPECT`.
4. If the FIT differs, recompose the device tree from the FIT and the overlays, and diff it against the analysed image and against v26.0.
5. Print the kernel `SRCREV` and the `.config` md5 from the build record chosen by `RECORD_TAG`.

Checks. Coded:

- **Stops** (exit 1) if `mmc0` is not the bring-up image, if the dry run flashes another wic, or if `jailhouse.dtbo` is not among the overlays.
- **Notes only:** a wic md5 that differs from the record ("ask before flashing"), and a FIT md5 that differs (the device tree is recomposed and diffed). The `.config` md5 is printed for comparison, not compared.

Expected output. The 700, refinement-1 image:

```
rity.json "mmc0": "rity-bringup-image-genio-700-evk.wic.img"
OK: mmc0 is the bring-up image
        name:     Rity Bringup Image (rity-bringup-image)
        codename: scarthgap
        overlays: ['video.dtbo', 'apusys.dtbo', 'display-dsi.dtbo', 'display-hdmi.dtbo',
'jailhouse.dtbo']
flashing mmc0=rity-bringup-image-genio-700-evk.wic.img
[...]
OK: the dry run flashes the bring-up wic
OK: jailhouse.dtbo in the overlays
wic rity-bringup-image-genio-700-evk.rootfs-20261023055822.wic.img md5 0e5cdeda ([...] hand-off:
0e5cdeda)
FIT fitImage--6.6.152+git0+45b3ad996a-r0-genio-700-evk-20261023055822.bin md5 b6f4aca8 (analysed:
b6f4aca8)
kernel SRCREV 45b3ad996aa04ec748806752d03b72d1908e87cf, .config md5
b793b281f18191314f1105be204f62fa, patches applied: 4, cfg fragments: 23
=== OK to flash
```

Last run. Pass on both boards, 2026-10-26: wic `240de91e` and FIT `03cd6d65` on the 510, wic `0e5cdeda` and FIT `b6f4aca8` on the 700 (board logs 175 and 183).

Sources. `rity-precheck.sh`; board log 183, board log 175.

IMG-02 First boot of a new image

Purpose. After a flash, the board must boot with what Jailhouse needs:

- the hypervisor's memory hole and the ivshmem reservation;
- the overlay in the boot configuration;
- the device tree unchanged for the devices that the cells use;
- the expected kernel options;
- no kernel warnings;
- the Jailhouse files installed.

A difference found here explains a later test failure before it happens.

Procedure. `first-boot.sh <prefix> <reference dir>`:

1. `capture.sh` saves the board's state: kernel, command line, memory map, environment, device tree, kernel configuration and kernel log.
2. Print the kernel, the boot arguments and the overlay variables of the environment.
3. Print `/proc/iomem` around the Jailhouse windows, beside the reference.
4. Diff the live device tree against the reference: `memory`, `reserved-memory`, `chosen`, and every node the cells use (UARTs, EINT, AFE, pin controller, clock controller, watchdog, PCIe, PSCI, CPUs, GIC, timer). Then the whole tree.
5. Print the Jailhouse fragment's kernel options beside the reference.
6. Count the kernel warnings, and list the image's Jailhouse files, cells and daemon unit.
7. Unless `NO_ENABLE=1`: `modprobe jailhouse` and `jailhouse enable`, with the console.

Checks. No coded verdict. The criteria (the RITY move notes):

- System RAM up to `6abfffff` and again from `6bb00000`, with `6ba00000-6bafffff` reserved.
- `boot_conf` ends in `#conf-jailhouse.dtbo`.
- The device-tree diff shows only the expected changes; for the refinement-1 image, none against the heads image.
- The kernel options are those of the configuration under test.
- 0 kernel warnings.
- The enable, where run, returns 0 with the expected hypervisor version.

Expected output. The 700, refinement-1 image, against the first boot of the heads image (`NO_ENABLE=1`), abridged:

```
boot_conf=#conf-mediatek_mt8390-genio-700-evk.dtb#conf-video.dtbo#conf-apusys.dtbo#conf-display-
dsi.dtbo#conf-display-hdmi.dtbo#conf-jailhouse.dtbo
[...]
61800000-6abfffff : System RAM
  66fb0000-67d5ffff : reserved
6ba00000-6baffff : reserved
6bb00000-23fffffff : System RAM
[...]
=== DT diff against [...] (memory, reserved-memory, chosen, and the devices our cells use)
0 differences in 706 properties
--- whole-tree count:
0 differences in 4826 properties
=== kernel config: Jailhouse fragment options
[...]
HOTPLUG_PCI_PCIE: # CONFIG_HOTPLUG_PCI_PCIE is not set (ref: CONFIG_HOTPLUG_PCI_PCIE=y)
[...]
MTK_JH_REMOTEPROC: CONFIG_MTK_JH_REMOTEPROC=m (ref: CONFIG_MTK_JH_REMOTEPROC=y)
[...]
=== kernel warnings this boot: 0 (ref 0)
=== Jailhouse files in the image
688a5889 /usr/lib/firmware/jailhouse.bin
[...]
mtk-v1.0.0 (28-ge7fd8796)
[...]
=== NO_ENABLE: Jailhouse left unloaded
```

Last run. As the criteria on both boards, 2026-10-26, refinement-1 image (board logs 177 and 185). The device tree was identical to the heads image's first boot: 0 of 4753 properties differ on the 510, 0 of 4826 on the 700.

Sources. `first-boot.sh` , `capture.sh` , `dt-diff.py` ; board log 185, board log 177.

IMG-03 Acceptance of the kernel fragment trim

Purpose. Refinement 1 turns off PCIe hotplug and builds the Jailhouse remoteproc driver and the rpmsg character devices as modules. The image must still work without manual steps:

- the remoteproc driver must load when Jailhouse is enabled;
- the rpmsg modules must load when the channel comes up;
- the rpmsg device nodes must stay root-only;
- pciehp must be gone, without harm to the PCIe port.

Procedure. `refine-accept.sh <board> <prefix>` , on a freshly flashed image with UART1 connected:

1. Print the kernel options and the installed modules. List the modules before the enable, then `jailhouse enable` . List the modules and the remoteprocs, and the kernel's lines for the ivshmem device.
2. Run REL-01's `rpmsg-sample.sh` with no manual `modprobe` , and print the device nodes and the modules.
3. Count `pciehp` in the kernel log and in `/sys/module` , and list the PCI Express port services.
4. Disable Jailhouse, unload it, and count the kernel warnings.

Checks.

- **Coded:** `mtk_jh_rproc` is loaded after the enable; `rpmsg-test` returns 0 and at least one message from Linux arrives.

- **Not coded** (the RITY move notes): before the enable, none of `jailhouse`, `mtk_jh_rproc`, `rpmsg_char` and `rpmsg_ctrl` is loaded; the rpmsg nodes are `crw------- root root`; no pciehp, with the port services `aer` and `pcie_pme`; 0 kernel warnings.

Expected output. The 700, abridged:

```
=== 1. before enable
(none of jailhouse, mtk_jh_rproc, rpmsg_char, rpmsg_ctrl loaded)
enable rc=0
mtk_jh_rproc           16384  0
jailhouse              45056  1
[...]
[  124.150056] remoteproc remoteproc1: 0001:00:00.0 is available
[  124.150088] mtk-jh-rproc 0001:00:00.0: peer 0 of 2, IPC-only: never loads or starts the remote
[...]
PASS  mtk_jh_rproc autoloaded on enable
=== 2. rpmsg sample, no manual modprobe
[...]
crw------- 1 root root 507, 0 Jun 26 08:46 /dev/rpmsg0
crw------- 1 root root 506, 0 Jun 26 08:46 /dev/rpmsg_ctrl0
rpmsg_ctrl             12288  0
rpmsg_char             20480  3 rpmsg_ctrl
[...]
rpmsg-test rc=0
destroy rc=0
=== UART1: 9 messages from Linux
PASS  rpmsg sample without modprobe
[...]
=== 3. pciehp
pciehp lines in dmesg: 0
aer pcie_pme
pciehp module entries: 0
[...]
disable rc=0
kernel warnings 0
```

Last run. All pass on both boards, 2026-10-26, refinement-1 image (board logs 178 and 186).

Sources. `refine-accept.sh`; board log 186, board log 178.

IMG-04 Permissions of the Jailhouse interfaces (review check B5)

Purpose. Whoever can open `/dev/jailhouse`, write a remoteproc's state, open an rpmsg device or edit the daemon's configuration controls the cells. On the image, all of these must be root-only. The security review asked for this check (B5).

Procedure. `b5-permissions.sh`, which holds the commands of board log 174 since 2026-10-26:

- `b5-permissions.sh board <prefix>`, on the board, read-only: with no cell, the module is loaded once if it was not, and unloaded again; Jailhouse is not enabled;
- `b5-permissions.sh image <rootfs.ext4> <prefix>`, on the host, from the extracted root file system of the flashed wic, read with `debugfs`, without a board. It also lists the mode of every cell file.

Both list:

- the device nodes and udev rules for `jailhouse`, `rpmsg`, `remoteproc` and `misc`;
- the sysfs controls;
- `/etc/jailhouse` and the daemon's unit;

- the files that root loads;
- the accounts.

Checks. No coded verdict. The criteria (the RITY move notes, Review check B5):

- `/dev/jailhouse` is `crw------- root root` and exists only while the module is loaded; no udev rule changes it.
- The remoteproc `state` and `firmware` files are 0644 root.
- `/etc/jailhouse` is absent as flashed.
- The daemon's unit runs as root, with `ConditionPathExists` on its configuration.
- Only root has a shell.
- The rpmsg nodes, 0600 root, were confirmed later by IMG-03.

Expected output. The 510, abridged:

```
=== modprobe jailhouse (not enabled)
crw------- 1 root root 10, 259 Jun 26 14:30 /dev/jailhouse
[...]
-r--r--r-- 1 root root 4096 Jun 26 14:30 /sys/devices/jailhouse/console
-r--r--r-- 1 root root 4096 Jun 26 14:30 /sys/devices/jailhouse/enabled
[...]
=== sysfs controls
-rw-r--r-- 1 root root 4096 Jun 26 14:30 /sys/class/remoteproc/remoteproc0/firmware
-rw-r--r-- 1 root root 4096 Jun 26 12:07 /sys/class/remoteproc/remoteproc0/state
[...]
=== /etc/jailhouse
ls: cannot access '/etc/jailhouse': No such file or directory
[...]
=== the daemon unit
[Unit]
Description=Restart failed Jailhouse cells
ConditionPathExists=/etc/jailhouse/cell-restart.conf
[...]
User=
```

Last run. As the criteria, 2026-10-22, with the commands run by hand: the 510 on the board, the 700's image on the host (board log 174). The notes record one informational finding: the hypervisor console in sysfs is world-readable. The script's board mode has not run since it was written.

Sources. `b5-permissions.sh`; board log 174.

06
Host-side checks

These checks run on the build host, without a board:

- the IoT Yocto integration: the series that moves Jailhouse into meta-mediatek-bsp and meta-rity, behind the `jailhouse` distro feature;
- the hypervisor sources and cells;
- the Zephyr image set.

The Yocto checks are recorded in the IoT Yocto integration record, which quotes their outputs.

HOST-01 Feature off equals the baseline

Purpose. With the `jailhouse` feature off, the series must change nothing: a customer who does not use Jailhouse gets the stock image.

Procedure. Build `rity-bringup-image` for `genio-510-evk` twice: at the layer heads (the baseline), and at the series without the feature. Compare the kernel `.config`, the kernel patches (by patch ID), the kernel fragments, the deployed overlays, the FIT configurations and the package manifest.

Checks. Every item is identical. Hash differences of the kernel, FIT and wic must be explained.

Expected output. The record's table (record section 5), abridged:

Item	Result
Kernel `.config`	identical (md5 `48c045630d3cddce906febc5ca41a428`, diff empty)
Kernel patches on top of `SRCREV` (patch-id)	identical: the same 2, no Jailhouse patch
Kernel `.cfg` fragments	identical: the same 22, no `6.6-jailhouse-vPCI.cfg`
Deployed dtbo set	identical: 32 overlays, byte-identical, no `jailhouse.dtbo`
FIT configurations	identical (33)
Rootfs manifest	identical (1711 packages), no jailhouse

Last run. Pass, 2026-10-21, `genio-510-evk`. The kernel hash difference is explained by HOST-02. The full comparison is in `identity-baseline-vs-featureoff-genio-510-evk.txt`.

Sources. Integration record, section 5.

HOST-02 Reproducibility of the stock kernel

Purpose. HOST-01's kernel hashes differ. The test shows that the difference comes from the build itself, not from the series.

Procedure. `stock-kernel-repro.sh`: build the stock kernel a second time from identical inputs (`bitbake -f -c compile linux-mtk`). Compare its image with the baseline's, and HOST-01's two kernels with each other.

Checks. The two stock builds differ as much as the baseline and the feature-off build do: same `.config`, patches, banner and size, different bytes.

Expected output. The record's table, from the record's output `repro-stock-kernel.txt`, abridged:

	baseline vs second stock build	baseline vs feature off
`.config`	identical (`48c04563…`)	identical
Image size	70302208 = 70302208	70302208 = 70302208
Image md5	`81d69690…` vs `3374f6fd…`: **different**	`81d69690…` vs `975ec5fd…`: different
Differing bytes	18706552	18694612
`strings -n 6`, sorted, differing lines	330	336

Last run. Pass, 2026-10-22. The cause is BTF: the heads encode it with pahole v1.25 in parallel, which emits types in no fixed order.

Sources. Integration record, section 5.

HOST-03 Feature on: the contents of the image

Purpose. With the feature on, the image must contain exactly the Jailhouse parts:

- the kernel patches and fragment;
- the overlay in the FIT;
- the hypervisor, module, cells, daemon and unit;
- the expected hypervisor revision.

Nothing else may change.

Procedure. Build `rity-bringup-image` with `DISTRO_FEATURES:append = " jailhouse"` for each machine. Check the patches, fragments, `.config` against feature off, version, hashes, overlay, packages, root file system, the revision `AUTOREV` resolved to, and U-Boot. The output is in `check-featureon-<machine>.txt`.

Checks.

- 4 kernel patches: the 2 stock ones, the Jailhouse exports and the `mtk_jh_rproc` driver.
- The `.config` differs from feature off only in the fragment's options.
- `jailhouse.dtbo` is deployed and in the FIT.
- The packages `jailhouse` and `kernel-module-jailhouse` are installed.
- The root file system has the hypervisor, the 9 cells, the daemon and the enabled unit.

Expected output. The record's table for `genio-700-evk`, abridged:

Check	Result
Kernel patches	4 = the 2 stock ones + `Added Jailhouse exports and prevent switch to VHE.` + the `mtk_jh_rproc` driver
Kernel fragment	`6.6-jailhouse-vPCI.cfg` applied (23 fragments)
Overlay	`jailhouse.dtbo` deployed; `conf-jailhouse.dtbo` in the FIT; `genio-flash --list-dtbo` lists it
Packages	`jailhouse 1.1+git0+e7fd8796d5-r0`, `kernel-module-jailhouse` (1714 packages)
`AUTOREV` resolved to	`mtk-genio-dev e7fd8796d5fc`, `git describe mtk-v1.0.0-28-ge7fd8796`

Last run. Pass for both machines, 2026-10-21.

Sources. Integration record, section 6; `check-featureon-genio-700-evk.txt`, `check-featureon-genio-510-evk.txt`.

HOST-04 The guard: Jailhouse in the image without the feature

Purpose. A configuration that adds `jailhouse` to the image without turning the feature on must fail at planning, with a clear reason. It must not build a half-configured image. The same holds on a machine that Jailhouse does not support.

Procedure. Parse-only runs (`bitbake -n rity-bringup-image`), with `IMAGE_INSTALL:append = " jailhouse"`:

1. on `genio-510-evk` without the feature;
2. on `genio-1200-evk`, with and without the feature.

Checks. The build is refused at planning. On the 510 the reason comes from `features_check`; on the 1200, from `COMPATIBLE_MACHINE`.

Expected output. Case 1:

```
ERROR: Nothing RPROVIDES 'jailhouse' (but [...]/rity-bringup-image.bb RDEPENDS on or otherwise
requires it)
jailhouse was skipped: missing required combined feature 'jailhouse' (not in COMBINED_FEATURES)
[...]
ERROR: Required build target 'rity-bringup-image' has no buildable providers.
[...]
EXIT CODE: 1
```

Case 2:

```
jailhouse was skipped: incompatible with machine genio-1200-evk (not in COMPATIBLE_MACHINE)
[...]
EXIT CODE: 1
```

Last run. Pass, 2026-10-21 (case 1) and 2026-10-22 (case 2).

Sources. Integration record, section 7; `guard-510.log`, `guard-1200-off.log`, `guard-1200-on.log`.

HOST-05 Feature on, on a machine without Jailhouse

Purpose. A distribution that turns the feature on for all its machines must build the others unchanged. The gated lines must expand to nothing where the machine feature is missing.

Procedure. `MACHINE = "genio-1200-evk"` with the feature on: `bitbake -e` of `linux-mtk`, `dtbo` and `rity-bringup-image`. Print `COMBINED_FEATURES`, `DISTRO_FEATURES`, `MACHINE_FEATURES`, `SRC_URI` and `IMAGE_INSTALL`.

Checks. `jailhouse` is in `DISTRO_FEATURES` but not in `COMBINED_FEATURES`, and none of the `SRC_URI`s or `IMAGE_INSTALL` contains anything of the series.

Expected output. Abridged:

```
== linux-mtk
COMBINED_FEATURES="usbgadget wifi bluetooth pci usbhost alsa"
DISTRO_FEATURES="[...] jailhouse [...]"
[...]
SRC_URI="git://git@gitlab.com/mediatek/aiot/bsp/linux.git;protocol=https;branch=mtk-v6.6 file://
disable-buildin-ax88179.cfg [...]"
[...]
EXIT CODE: 0
```

Last run. Pass, 2026-10-21.

Sources. Integration record, section 7.

HOST-06 Stale overlay after switching the feature off

Purpose. In a build directory that once built the feature on, the BSP's `dtbo` recipe compiles every leftover `*.dts`. So a later build with the feature off would still ship `jailhouse.dtbo` in its FIT. The fix removes the leftover; the test proves that it does, and that it changes nothing with the feature on.

Procedure. `fixtest.sh`, in a separate build directory with its own `TMPDIR` and hash-equivalence database. `genio-510-evk` with the fix, first with the feature on, then off. Compare the off build with the stock baseline, and the hash-equivalence records of the two builds.

Checks.

- The dtbo work directory loses `jailhouse.dts` and `jailhouse.dtbo`.
- The off build equals the baseline in content: the same `.config`, 32 overlays, 33 FIT configurations, the same manifest.
- The two builds keep separate output hashes.
- The on build is unchanged.

Expected output. `fix-dtbo-workdir.txt`, abridged:

```
== after feature ON (2026-10-22 23:55:19)
-rw-r--r-- 1 mtk mtk  623 23:33:12 [...]/tmp/work/genio_510_evk-poky-linux/dtbo/6/build/
jailhouse.dtbo
-rw-r--r-- 1 mtk mtk 1259 22:14:04 [...]/tmp/work/genio_510_evk-poky-linux/dtbo/6/jailhouse.dts
== after feature OFF (2026-10-23 00:34:50)
ls: cannot access '[...]/tmp/work/genio_510_evk-poky-linux/dtbo/6/jailhouse.dts': No such file or
directory
ls: cannot access '[...]/tmp/work/genio_510_evk-poky-linux/dtbo/6/build/jailhouse.dtbo': No such
file or directory
== feature-off log.do_compile (dtbo): jailhouse lines
DEBUG: Executing python function jailhouse_drop_stale_overlay
DEBUG: Python function jailhouse_drop_stale_overlay finished
```

The content comparison with the baseline is in the record's comparison `identity-baseline-vs-fix-off-genio-510-evk.txt`.

Last run. Pass, 2026-10-22 23:13 to 2026-10-23 00:35. The fix is not part of the initial series.

Sources. Integration record, sections 9.1 and 11; `fix-dtbo-workdir.txt`.

HOST-07 Hypervisor sources compile cleanly

Purpose. The hypervisor's and inmates' sources must compile without warnings under the hypervisor's own strict flags (`-Werror -Wall -Wextra` and more). A quick check is possible without a kernel tree.

Procedure. `hvcheck.sh [-i] <file.c>...`: generate the version and assembler-offset headers in a temporary directory, removed at the end, then run `aarch64-linux-gnu-gcc -fsyntax-only` with the hypervisor's flags (or the inmates' with `-i`) on each file. This is a syntax and warning check, not a build. Since 2026-10-26 the output is also kept in a board log `hvcheck-<date>.log`.

Checks. Coded: `OK <file>` per file. The exit status is non-zero if any file fails.

Expected output. `hvcheck-20261026-132415.log`, paths shortened:

```
=== Mon Oct 26 01:24:15 PM PDT 2026 mtk-v1.0.0-28-ge7fd8796 [...]/mtk-common-wdt.c [...]/mtk-
mt8188-wdt.c
OK  [...]/hypervisor/arch/arm64/mtk-common-wdt.c
OK  [...]/hypervisor/arch/arm64/mtk-mt8188-wdt.c
```

Last run. Pass, 2026-10-26, on the two watchdog-unit sources at `e7fd8796`. The W1 milestone notes record the release gates of 2026-10-12, which cover all sources: every commit builds with and without `CONFIG_SOC=mt8188`, with 0 warnings.

Sources. `hvcheck.sh`; the board log `hvcheck-20261026-132415.log`.

HOST-08 Cell configurations: build and `jailhouse-config-check`

Purpose. Every cell configuration must compile and pass Jailhouse's configuration checker together with its root cell. The checker finds overlapping memory, missing devices and invalid entries before a board does.

Procedure. `cellcheck.sh <build dir> <510|700> [more .cell]`, on a built hypervisor tree:

1. Take the board's root cell and every other cell that the build made from the board's cell table (`configs/arm64/genio-<board>-evk-cells.c`, since release 1.1), plus any further cells given, such as a test cell.
2. Print their md5, and run `tools/jailhouse-config-check` on all of them together.

The output is also kept in a board log `cellcheck-<board>-<build>-<date>.log`. Until 2026-10-26 the script compiled one `.cell` per `.c` file, which no longer matches the cell tables.

Checks. Coded: the checker's exit status is the script's (`rc=0`).

Expected output. `cellcheck-700-e7fd8796-700-20261026-132406.log`, abridged, paths shortened:

```
=== Mon Oct 26 01:24:06 PM PDT 2026 mtk-v1.0.0-28-ge7fd8796, genio-700-evk
25ca9f48 [...]/configs/arm64/genio-700-evk.cell
[...]
a9bee36a [...]/test-ok.cell
Reading configuration set:
  Architecture:  arm64
  Root cell:     genio-700-evk ([...]/genio-700-evk.cell)
  Non-root cell: uart-demo ([...]/genio-700-evk-uart-demo.cell)
[...]
  Non-root cell: test-ok ([...]/test-ok.cell)
Overlapping memory regions inside cell: None
Overlapping memory regions with hypervisor: None
Missing PCI MMCONFIG interceptions: None
Missing resource interceptions for architecture arm64: None
rc=0
```

Last run. Pass, 2026-10-26, on the 700's cells of the build `e7fd8796` and the test cell `test-ok`. For the 510 the last record is the W1 milestone notes' gate of 2026-10-12: `jailhouse-config-check` (revision 16) clean for both boards.

Sources. `cellcheck.sh`; the board log `cellcheck-700-e7fd8796-700-20261026-132406.log`.

HOST-09 Zephyr build sets

Purpose. Every sample and test image that the boards run must build for every Genio target from one recorded tree, without warnings. The board results then apply to a known set of images.

Procedure. `twister` builds (`--build-only`) of the rel-1.1 tree (mtk-zephyr `2e9e70f0a16`, samples `b962ff3`, SDK 1.0.1):

- `twister-samples`: all samples, `hello_world`, the rpmsg sample and the expiry sample, for both A55 targets;
- `twister-all`: the samples, `hello_world`, `gpio_basic_api`, the SMP tests, `pi` and `synchronization`, for all four targets.

Checks. Every configuration builds, with 0 errors and no warnings.

Expected output. The ends of `twister-samples.out` and `twister-all.out`:

```
INFO    - 0 of 38 executed test configurations passed (0.00%), 38 built (not run), 0 failed, 0
errored, with no warnings in 596.46 seconds.
```

```
INFO    - 0 of 92 executed test configurations passed (0.00%), 92 built (not run), 0 failed, 0
errored, with no warnings in 1296.96 seconds.
```

"0 of N passed" is twister's wording for build-only runs: nothing was executed.

Last run. Pass, 2026-10-21.

Sources. The record of the Zephyr image set; `twister-samples.out`, `twister-all.out`.

HOST-10 Zephyr image manifest and comparison with the tested images

Purpose. The board runs of release 1.1 on 2026-10-12/13 used images built before the final tree. The test shows that the final images are the same content, so those runs stand for them.

Procedure. Record the md5 of every image in `MANIFEST.md5`. Compare the set, with `compare.py`, against the set that ran on the boards (`uart-dma-en`), ignoring the build hash and twister's run ID.

Checks. Every image with a counterpart is identical apart from the build hash, and the images without one are explained.

Expected output. `compare.txt`:

```
rel-1.1 against [...]/uart-dma-en
identical apart from the build hash (and the twister RunID): 141
different: 0
no counterpart there: 6
```

The six without a counterpart are builds of the expiry sample, which landed after that set. `rpmsg-test` is byte-identical to the tested binary (`ecd8dac7`).

Last run. Pass, 2026-10-21.

Sources. The record of the Zephyr image set; `compare.txt`, `MANIFEST.md5`.

07
Status of the scripts

7.1 Verdicts
7.2 Open points
7.3 Expected output not captured in the logs
7.4 Script changes of 2026-10-26

7.1 Verdicts

Only some scripts decide pass or fail themselves. For the others, a person reads the output against the criteria in each entry's Checks field. Chapter 8 proposes coded criteria for several of them.

Tests	Script	Verdict
R-01 to R-52	`regression.sh`	Coded: `PASS` or `FAIL` per check, a summary and the exit status
W1, all 33	`rity-w1.sh`	None: observations, judged against the W1 milestone notes. W1-M3-T18 prints an expected- `EBUSY` line of its own
REL-01, REL-02	`rpmsg-sample.sh`, `genio-inmate-test.sh`	None
REL-03 to REL-05	`extras.sh`	None
REL-06	`no-overlay.sh`	None. It stops only if the overlay entry is not found exactly once, or if the board does not come back
REL-07, REL-08	`reset-genio.sh`	None. It reports a board that does not come back within 240 s
REL-09, REL-10	`pcie-test.sh`, `sr2-test.sh`	None
REL-11	`fixes2-test.sh`	Coded: `good` or `BAD` per reload, and the count
REL-12	`fixes2-test.sh`	None
IMG-01	`rity-precheck.sh`	Coded: it stops on three conditions. The hash comparisons are notes
IMG-02	`first-boot.sh`	None
IMG-03	`refine-accept.sh`	Coded for two of its three steps. The module baseline, the device node modes and the pciehp step are not coded
IMG-04	`b5-permissions.sh`	None
HOST-01 to HOST-06	build records	None: comparisons recorded in the integration record
HOST-07, HOST-08	`hvcheck.sh`, `cellcheck.sh`	Coded: the exit status
HOST-09	`twister`	Coded: twister's summary and exit status
HOST-10	`compare.py`	Counts of identical, different and unmatched images

7.2 Open points

- **Not run since a change.**
  - REL-10's case T5 has not run with `hwinfo_api` as the restart image.
  - `b5-permissions.sh` has not run in its board mode (IMG-04).
  - W1-TWO, W1-M3-T18 and REL-03 to REL-05 ran again on the 700 only.
  - The regression's new captures (R-21, R-45, R-46) await its next run (Section 7.3).
- **`rity-precheck.sh` (IMG-01).** It stops only on the image name, the dry run and the overlay. The wic and FIT hashes are reported, not enforced.
- **`first-boot.sh` (IMG-02).** It compares the kernel configuration with the reference boot, not with the build record. The check that no Jailhouse module is loaded before the enable is IMG-03's step 1.
- **`pcie-test.sh` (REL-09).** On this BSP it does not exercise the root path of `SWSYSRST`, as its header now says. W1-M2-T10 covers that path with direct writes.
- **Host-specific paths.** Fixed in the scripts of this catalogue. One-off scripts of 2026-10-12 and 2026-10-13, which are not in the catalogue, still read a temporary directory.

7.3 Expected output not captured in the logs

Test	What is missing
R-21	The unfiltered output `psci-probe-deny.raw`. The script keeps it since 2026-10-26, and no run has produced it yet. The output itself, `A+-`, is in `uart1.log`
R-45	The error message of `jailhouse cell create` for `test-mapped`. Since 2026-10-26 the script prints it as the detail of the `PASS` line; no run has produced it yet. The hypervisor console has no line for the refusal
R-46	The error message of `jailhouse cell create` for `test-busy`, as for R-45

7.4 Script changes of 2026-10-26

Revision 1.0 of this catalogue listed defects in the scripts. These changes followed, as the script-fix record lists them.

Item	Change	Evidence
W1-TWO, case 2	`rity-w1.sh` and `m3-two.sh` load `wdt-probe-4-nouart.bin` into `wdt-b`, and the console check also shows `Unhandled` lines	Board log 188
W1-M3-T18	An expected- `EBUSY` check; no traceback	Board log 189
`m3-two.sh`	Pushes `wdt-b.cell` from where the test cells are now	–
REL-03 to REL-05	`extras.sh` repaired: its helpers folded in, the image's own files used, the refusal by a bind mount, Jailhouse left as found. New usage `extras.sh <board> <log prefix> [no-SoC jailhouse.bin]`	Board log 190
REL-09	`pcie-test.sh` 's header states that the rebind does not reach `SWSYSRST` on this BSP	–
REL-10	`sr2-test.sh` 's case T5 needs a restart image other than the warm-reboot application, names it, and keeps the daemon's configuration. The header records what log 136 showed	Not run since
REL-11, REL-12 and the regression's test cells	`pcie-test.sh`, `sr2-test.sh` and `fixes2-test.sh` take the adb transport from `ADB_T`, and read `poll-state.py`, the test tools and the REL-12 test cells from the workspace, not from a temporary directory. The sources of the test cells are recovered; their `build.sh` rebuilds all eight cells byte for byte	The script-fix record
HOST-07	`hvcheck.sh` keeps its generated headers in a temporary directory, and its output as a board log	`hvcheck-20261026-132415.log`
HOST-08	`cellcheck.sh` rewritten for the per-board cell tables	`cellcheck-700-e7fd8796-700-20261026-132406.log`
IMG-04	`b5-permissions.sh`, the commands of board log 174 as a script	Board mode not run since
R-21, R-42, R-45, R-46	`regression.sh` keeps the unfiltered UART output (`<name>.raw`), prints the refusal messages as the detail of R-45 and R-46, and R-42's comment is corrected	Next regression run

08

Appendix: proposed coded criteria

8.1 rity-w1.sh (Chapter 3)

8.2 first-boot.sh (IMG-02)

8.3 rity-precheck.sh (IMG-01)

8.4 refine-accept.sh (IMG-03)

8.5 Not covered

> NOTE
> This chapter is a proposal. Nothing in it is implemented: the scripts behave as the entries describe. It is input for the test specification of the sanity and unit tests.

The scripts below print observations only. For each test, the proposal names a check that the script could code from the output it already prints. The suggested form is that of `regression.sh`: a result line per check, a summary and an exit status, with the current output kept for reading. Time windows come from the spread of the board logs. The source is the coded-criteria proposal (Section 1.5).

8.1 `rity-w1.sh` (Chapter 3)

Tests	Proposed criterion
W1-M2-T01, W1-M3-T20	`MODE` 0x5 and `LENGTH` 0xf800 (T20: after the disable and after the enable); uptime advanced by at least 40 s; no `WDT: no tick` line
W1-M2-T02	Read-backs `MODE` 1 and `LENGTH` 0x1000 and five kicks; `failed` 6.5 to 8 s after the start, with "missed its deadline" and "watchdog timeout"
W1-M2-T03, W1-M3-T15	`failed` 2.5 to 4 s after the start with both CPUs; after the destroy, `online` equals `possible`. T15 also: `CPU_ON` returns 0, and `b` is printed
W1-M2-T04, W1-M3-T12	`failed` 0.5 to 2 s after the start, with "chip reset requested". T04 also: `R` after the wrong key, uptime monotonic. T12 also: the next cell prints `s40000000`
W1-M2-T05	AFE cell: read-backs `4000`, then `0`, and "may not reset subsystems 10000, ignored" exactly once. Plain cell: `0` and `0`, and `14000`. Both cells `running`
W1-M2-T06	`running` after 20 s without ticks; "no tick from the root cell" once; "ticks from the root cell resumed", then `failed` within 1 s
W1-M2-T07	`running` 6 s after the reload
W1-M2-T08, W1-M2-T09	Destroy, disable and enable return 0; the root cell only, with all CPUs
W1-M2-T10	Linux's bit 16 reads `0x10000`, then 0; bit 14 reads 0 while the AFE cell exists
W1-M3-T11	`s80000000`, `s00000000` and `s80000000` in that order; the `STA` write reads back unchanged; the lock bit 0 in the cell and in the hardware; `WDT_INTERNAL` 0 in the cell and `0xfff` from Linux before and after
W1-M3-T13	No `F`; `failed` with "system reset requested"; the next cell prints `s40000000`
W1-M3-T14	`ffffffff`; the cell `running`

Tests	Proposed criterion
W1-M3-T16	`failed` 2.5 to 4 s after the start
W1-M3-T17	"Unhandled data read at 0x10000000" and "Parking CPU"; `poll()` wakes within 0.5 s, state `failed`
W1-M3-T18	Already coded (Section 7.1); only the `PASS` / `FAIL` form is missing
W1-M3-T19	Skipped, not failed, if `MODE.EN` stays set. Otherwise: `MODE` 0x4; `failed` 2.5 to 4 s after the start; uptime advanced by at least 35 s; after the restore `RuntimeWatchdogUSec=31s`, `MODE` 0x5, `LENGTH` 0xf800
W1-Z-EXPIRY	`failed` 14.5 to 16 s after the arm line
W1-Z-TASKWDT	The four UART lines of the entry; `failed` 3 to 4.5 s after the start, with "system reset requested"
W1-Z-HWINFO	In each of the three runs: 3 passed, 1 skipped (`test_device_id_get`), and `PROJECT EXECUTION SUCCESSFUL`
W1-Z-WDTBASIC	Second boot: `SUITE PASS` and `PROJECT EXECUTION SUCCESSFUL`
W1-Z-WDTERR	16 passed, 7 skipped, and `PROJECT EXECUTION SUCCESSFUL`
W1-TWO	Case 1: `-zephyr failed` 2.5 to 4 s after the start, with "watchdog timeout", and `wdt-b running` 20 s later. Case 2: `wdt-b failed` with "watchdog timeout", **no failed CPU and no `Unhandled` line** after its start, and `-zephyr running`. The last two checks would have caught the error of 2026-10-22
W1-D0	Per cell type: "restarting", then "restarted" within 0.5 s; `s40000000` from probe 6; for `-zephyr-rpmsg`, "detached" between them. The proposal places "detached" before "restarting"; in the logs it follows "restarting" (W1-D0, W1-D4)
W1-D1	Restarts 1, 3, 7 and 15 s (±0.5 s) after failures 2 to 5; the sixth failure logs "failed 5 times within 300 s", and no restart follows
W1-D2	10 "restarted" lines, 3.8 to 4.5 s apart; the pool counters equal before and after; `online` equals `possible`
W1-D3	Restarted; `s00000000`
W1-D4	"Hello from Zephyr!" in both `rpmsg-test` runs; the state "detached" before and "attached" after the restart. The proposal also names `rpmsg-test`'s return code 0; the script stops `rpmsg-test` after 8 s, so it returns 124 (W1-D4), and the call or the criterion must change
W1-D5, W1-D6	`CAUSE 0x8` first, `PSCI_FEATURES ... = 0` twice; "system reset requested"; "restarted" in the journal; then `hwinfo_api`'s `PROJECT EXECUTION SUCCESSFUL`
The whole run	0 kernel warnings. The hypervisor log has exactly two `Unhandled` / `FATAL` pairs, both "Unhandled data read at 0x10000000" (W1-M3-T17 and W1-D3); any other address fails the run. On 2026-10-22, the third pair, at `0x11001214`, was W1-TWO's fault

The time windows need each test's start time beside its failure time. `poll-state.py` prints it for the cases that use `run`; the others do not yet.

8.2 `first-boot.sh` (IMG-02)

Step	Proposed criterion
Memory map	System RAM ends at `6abfffff`, another range starts at `6bb00000`, and `6ba00000-6bafffff` is reserved
Overlay	`boot_conf` ends with `#conf-jailhouse.dtbo`
Device tree	No differences against the reference outside an allow-list per image: none for refinement 1; for the heads image against v26.0, the UART2 DMA and t-phy properties
Kernel configuration	The fragment's options equal an expected list per image. The build record's `.config` md5 would be stricter than the reference boot
Warnings	0
Enable, unless `NO_ENABLE=1`	Return code 0; the expected hypervisor version; the four `Initializing unit: mt8188_*` lines

8.3 `rity-precheck.sh` (IMG-01)

Its three stops are coded already. The hashes would become stops too:

Item	Proposed criterion
wic md5	Equal to the build record chosen by `RECORD_TAG` (or `WIC_EXPECT`), else exit 1; an override variable for a deliberate other image
FIT md5	Equal to `FIT_EXPECT`, or else the recomposed device tree equals the analysed one; otherwise exit 1
`.config` md5	Equal to the build record's, else exit 1

8.4 `refine-accept.sh` (IMG-03)

Step	Proposed criterion
Module baseline	Before the enable, none of `jailhouse`, `mtk_jh_rproc`, `rpmsg_char` and `rpmsg_ctrl` is loaded. This needs a fresh boot, or `NO_ENABLE=1` at the first boot
Device modes	Each `/dev/rpmsg*` while the channel is up: `600 root root`
pciehp	No `pciehp` in the kernel log and no `/sys/module/pciehp`; the root port's services do not include pciehp. On the 700, the Wi-Fi still enumerates (`wlp1s0`)
End	0 kernel warnings after the disable and `rmmod`

8.5 Not covered

The proposal does not cover the other scripts that print only observations: `rpmsg-sample.sh`, `genio-inmate-test.sh`, `extras.sh`, `no-overlay.sh`, `reset-genio.sh`, `pcie-test.sh`, `sr2-test.sh` and `b5-permissions.sh`. They would take the same treatment; their criteria are in the Checks fields of their entries.

09
Test index

The index lists every entry in document order. An entry with several identifiers covers the instances named in it.

Test	Name
R-01, p. 10	Hypervisor enable
R-02, p. 11	Root pin state unchanged by the mediation
R-03, p. 11	Root cell's EINT interrupt enabled
R-04, p. 11	Bare-metal inmate uart-demo
R-05, p. 12	Zephyr hello_world
R-06, p. 12	Ten create, start and destroy cycles
R-07, R-08, p. 13	Memory window and GPIO loopback samples
R-09, p. 14	Boot and timer sample
R-10, R-11, p. 14	UART receive interrupt and runtime reconfiguration
R-12, p. 15	hello_world on the A78
R-13, p. 15	Five cycles on the A78, with the CPU back with Linux
R-14, R-15, p. 16	Memory window and GPIO loopback on the A78
R-16, p. 16	Boot and timer on the A78
R-17, p. 17	UART receive interrupt on the A78
R-18, p. 17	hello_world on the lowest CPU
R-19, p. 17	Five cycles of the SMP cell, with both CPUs back
R-20, R-21, p. 18	PSCI CPU_ON allowed and denied
R-22, p. 18	SMP synchronisation, ten starts
R-23, p. 19	Pending SGIs in the redistributor (sgi-probe)
R-24, p. 19	GPIO loopback in the SMP cell
R-25 to R-34, p. 20	Audio samples in -zephyr-afe
R-35 to R-40, p. 22	Audio in the AFE cells on the A78 and on two A55 cores
R-41, p. 23	IOCFG not mediated: Linux's write reaches a cell's pin
R-42, p. 23	Sub-word reads of a register without entries
R-43, p. 24	Root write carrying a cell's bit (EINT D0EN_1, fix J4)

Test	Name
R-44, p. 24	Root byte accesses to a mediated register (fix J5)
R-45, p. 25	Refusing a cell that maps a mediated window
R-46, p. 25	Refusing a cell that claims another cell's pin
R-47, p. 26	Pin returned to Linux after destroy
R-48, p. 26	Big cores: cell cycles and Linux hotplug
R-49, p. 26	Root write to a pin configuration register not trapped
R-50, p. 27	The hypervisor console of the run
R-51, p. 27	Disable and re-enable
R-52, p. 28	The hypervisor console after the re-enable
W1-M2-T01, p. 30	Linux's watchdog through the unit
W1-M2-T02, p. 31	Kick, then stop
W1-M2-T03, p. 31	Never kick, in the SMP cell
W1-M2-T04, p. 32	Software reset fails only the cell
W1-M2-T05, p. 32	Subsystem reset ownership
W1-M2-T06, p. 33	Ticks from Linux stopped and resumed
W1-M2-T07, p. 33	A reload disarms the watchdog
W1-M2-T08, W1-M2-T09, p. 34	Destroy and disable with the watchdog armed
W1-M2-T10, p. 34	Linux's subsystem resets and a cell's reset bit
W1-M3-T11, p. 35	Reset cause after a timeout, and register isolation
W1-M3-T12, p. 35	Reset cause after a software reset
W1-M3-T13, p. 36	PSCI SYSTEM_RESET fails the cell
W1-M3-T14, p. 36	SYSTEM_RESET in a cell without a watchdog
W1-M3-T15, p. 37	SMP cell, both CPUs busy
W1-M3-T16, p. 37	A fault inside the cell
W1-M3-T17, p. 37	A stage-2 fault fails the cell at once
W1-M3-T18, p. 38	A second shipped cell on UART1 is refused
W1-M3-T19, p. 38	The cells' watchdogs without Linux's
W1-M3-T20, p. 39	Linux's watchdog across disable and enable
W1-Z-EXPIRY, p. 40	Watchdog expiry sample
W1-Z-TASKWDT, p. 40	Task watchdog sample and reboot through PSCI
W1-Z-HWINFO, p. 41	Reset cause through hwinfo

Test	Name
W1-Z-WDTBASIC, p. 41	Zephyr's basic watchdog API test
W1-Z-WDTERR, p. 42	Zephyr's watchdog error-case test
W1-TWO, p. 42	Two cells with watchdogs at once
W1-D0, p. 44	The shipped unit restarts every cell type
W1-D1, p. 45	Back-off and restart limit
W1-D2, p. 45	Ten expiry and restart cycles
W1-D3, p. 46	Restart after a stage-2 fault
W1-D4, p. 46	The rpmsg cell: detach, restart and attach again
W1-D5, W1-D6, p. 47	Zephyr's warm and cold reboot
REL-01, p. 48	The rpmsg sample
REL-02, p. 49	The samples' cell launcher genio-inmate.sh
REL-03, p. 50	Refusal of the root cell by a hypervisor without the MediaTek units
REL-04, p. 51	Disable and enable cycles with the ivshmem device
REL-05, p. 52	hello_world in the rpmsg cell
REL-06, p. 52	Booting without jailhouse.dtbo, and restoring it
REL-07, p. 54	Hardware watchdog reset after a Linux crash
REL-08, p. 55	Plain reboot and the reset status
REL-09, p. 55	PCIe controller unbind and rebind with cells running (700)
REL-10, p. 56	PSCI SYSTEM_RESET2 (510)
REL-11, p. 58	In-place reload runs the new image (700)
REL-12, p. 59	The base of a vendor entry (700)
IMG-01, p. 60	Pre-flight gate
IMG-02, p. 61	First boot of a new image
IMG-03, p. 62	Acceptance of the kernel fragment trim
IMG-04, p. 63	Permissions of the Jailhouse interfaces (review check B5)
HOST-01, p. 65	Feature off equals the baseline
HOST-02, p. 65	Reproducibility of the stock kernel
HOST-03, p. 66	Feature on: the contents of the image
HOST-04, p. 67	The guard: Jailhouse in the image without the feature
HOST-05, p. 68	Feature on, on a machine without Jailhouse
HOST-06, p. 68	Stale overlay after switching the feature off

Test	Name
HOST-07, p. 69	Hypervisor sources compile cleanly
HOST-08, p. 69	Cell configurations: build and jailhouse-config-check
HOST-09, p. 70	Zephyr build sets
HOST-10, p. 71	Zephyr image manifest and comparison with the tested images